



POLITÉCNICA

"Ingeniamos el futuro"

CAMPUS  
DE EXCELENCIA  
INTERNACIONAL

MiW

# Patrones de Diseño

## 28. Ejercicios

*Luis Fernández Muñoz*

<https://www.linkedin.com/in/luisfernandezmunyoz>

[setillofm@gmail.com](mailto:setillofm@gmail.com)

## Códigos

*Todos los códigos ejemplo de los patrones se encuentran en:*

<https://github.com/miw-upm/IWVG/tree/master/doo/src/main/java/designPatterns>

*Todos los códigos de ejercicios sobre patrones se encuentran en:*

<https://github.com/miw-upm/IWVG/tree/master/doo/src/main/java/desingPatterns/exercises>

## INDICE

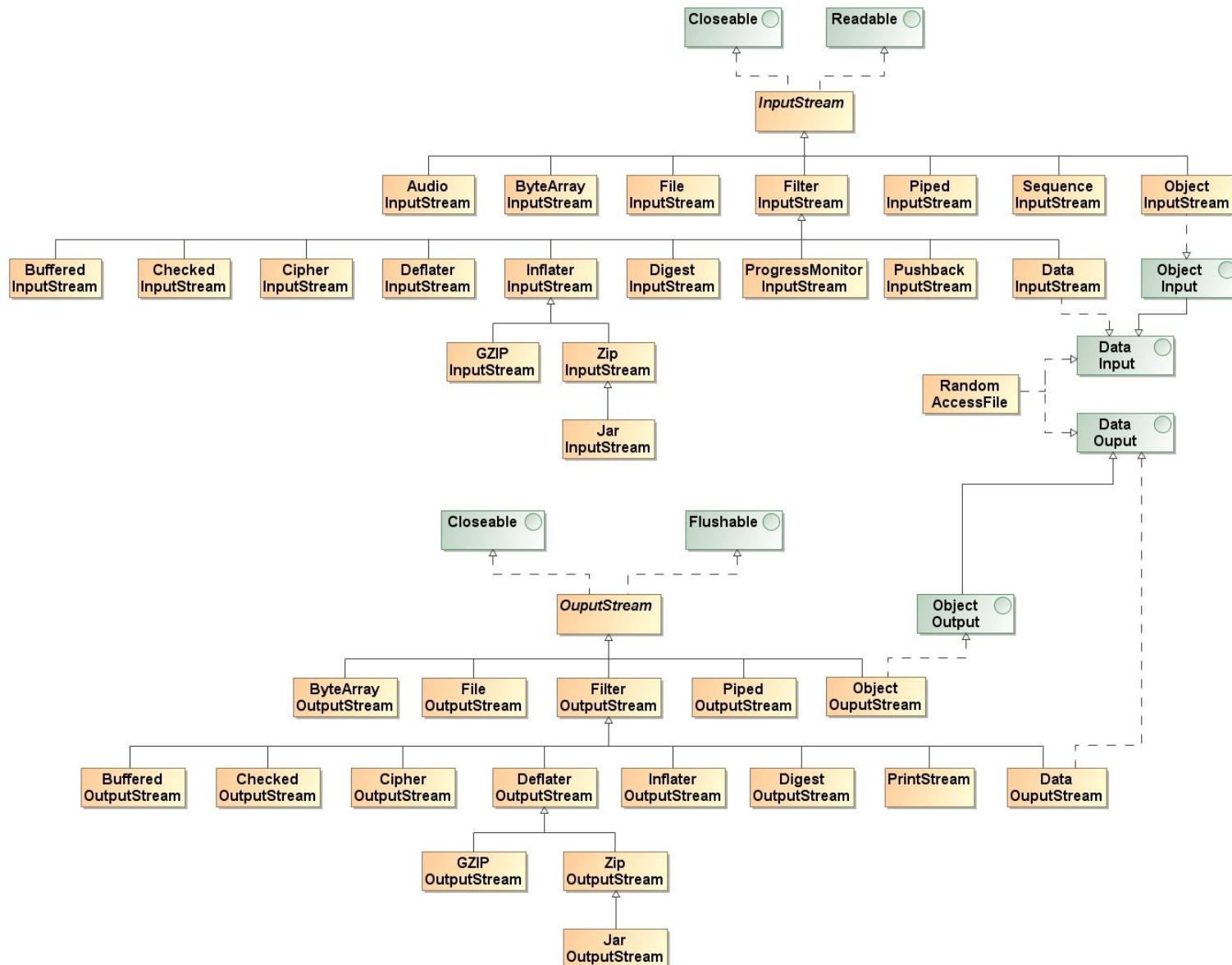
1. **Entrada/Salida en Modo Consola**
2. **Cadena Internacional de Supermercados**
3. **Gestor Local/Remoto OnLine/OffLine**
4. **Juego del Poker**
5. **Máquina de Venta de Café**
6. **Juego de las Tres en Raya**
7. **Script de Bombardeo**



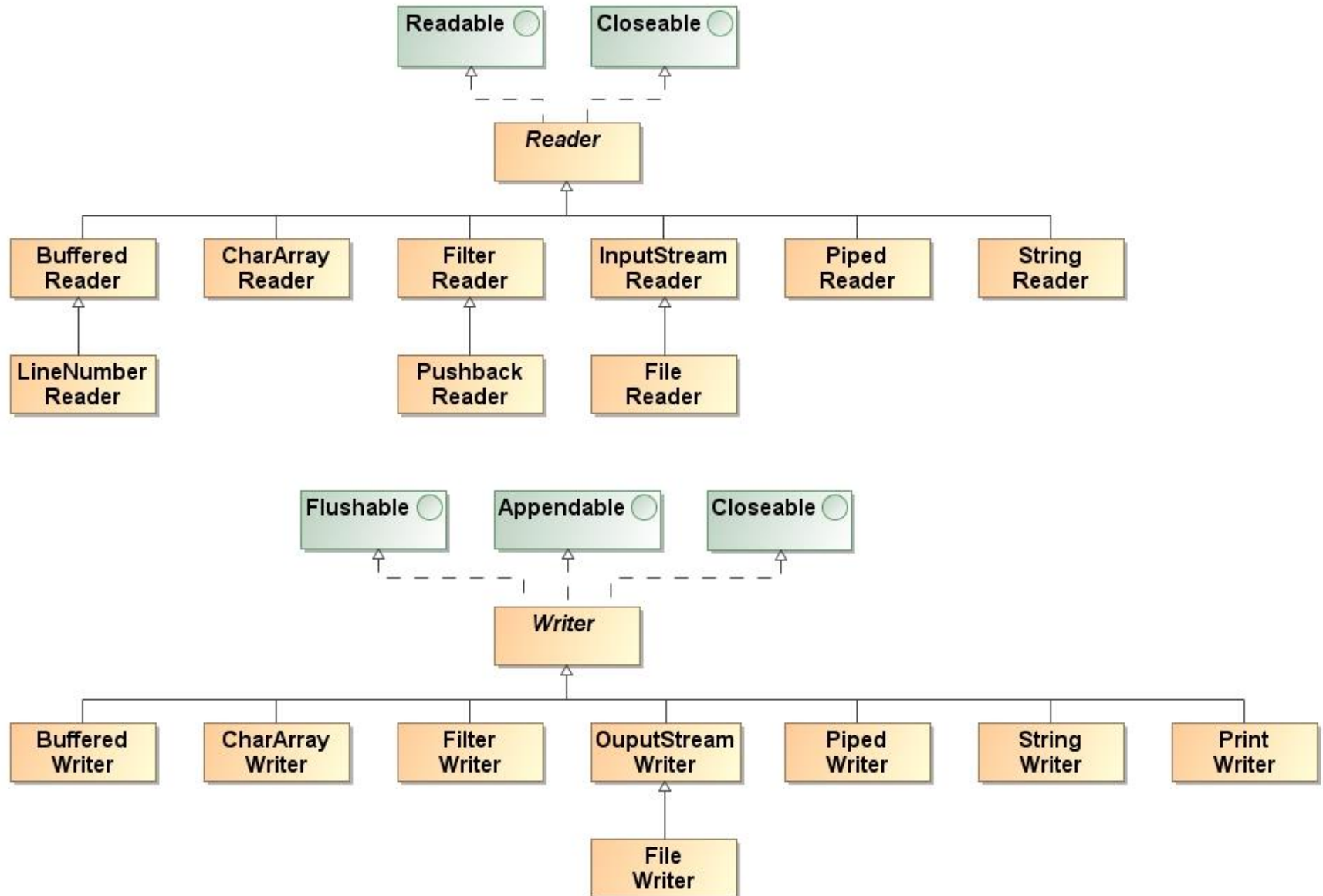
# 1. Entrada/Salida en Modo Consola

- **Contexto.** Los flujos (*streams*) son objetos que representan secuencias ordenadas de datos que tienen una fuente (flujos de entrada) o un destino (flujos de salida);
  - Las clases de entrada/salida del paquete java.io se definen en términos de flujos, y permiten a los programadores abstraerse de los detalles específicos del sistema operativo al acceder a los recursos del sistema, tales como ficheros, pantalla, teclado, líneas de comunicaciones, etc.;
  - En la biblioteca se distinguen dos grandes tipos de flujos:
    - de entrada y salida de bytes, cuyas clases abstractas de las que heredan todas las demás son *InputStream* y *OutputStream*;
    - de entrada y salida de caracteres<sup>1</sup>, cuyas clases abstractas de las que heredan todas las demás son *Reader* y *Writer*;

# 1. Entrada/Salida en Modo Consola

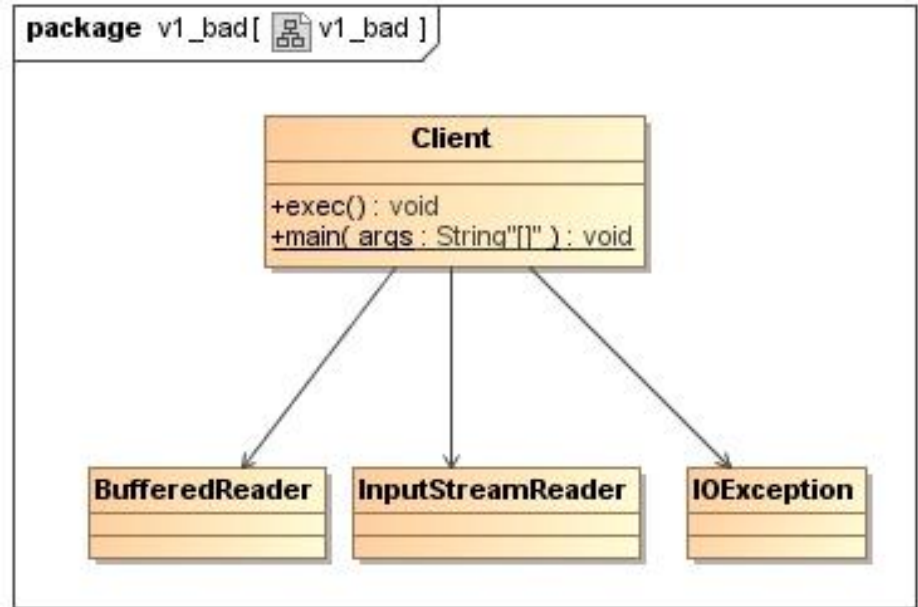


# 1. Entrada/Salida en Modo Consola



# 1. Entrada/Salida en Modo Consola

- **Enunciado.** Las posibilidades del subsistema son inmensas pero en la mayoría de las aplicaciones se requiere un uso muy particular respecto de todas las posibilidades.
  - Este uso se obtiene con la colaboración de objetos de varias clases del subsistema
  - Esto acopla y repite código en todos aquellos clientes que requieran las funcionalidades del subsistema



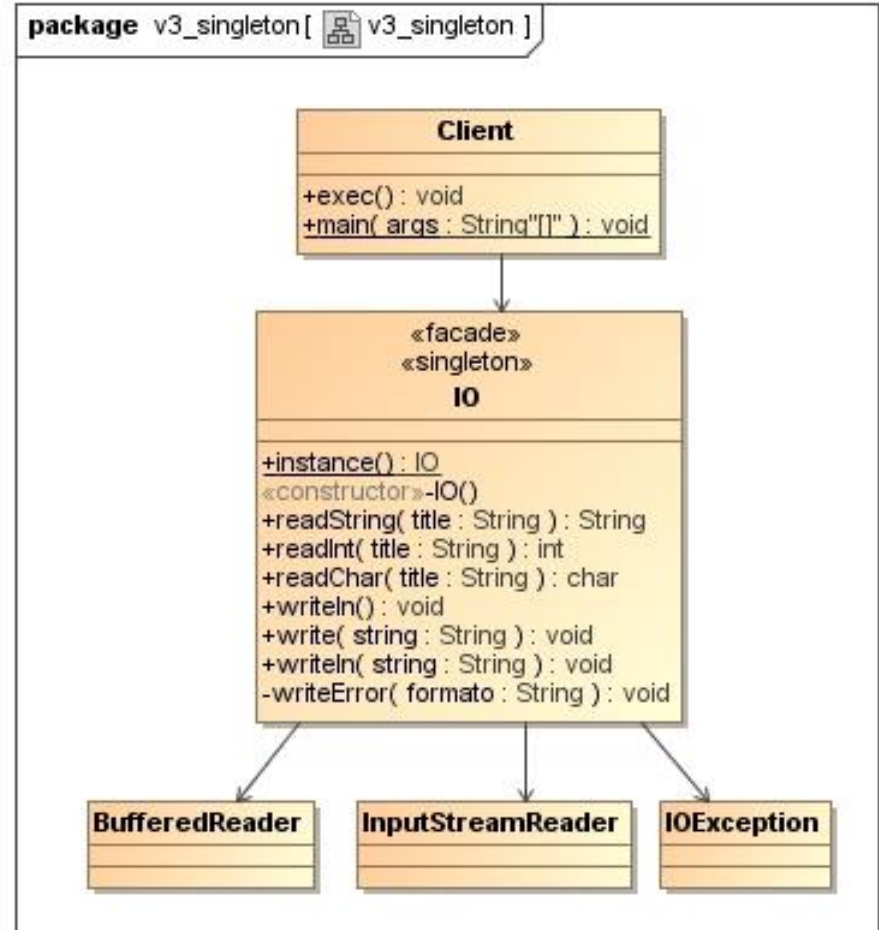
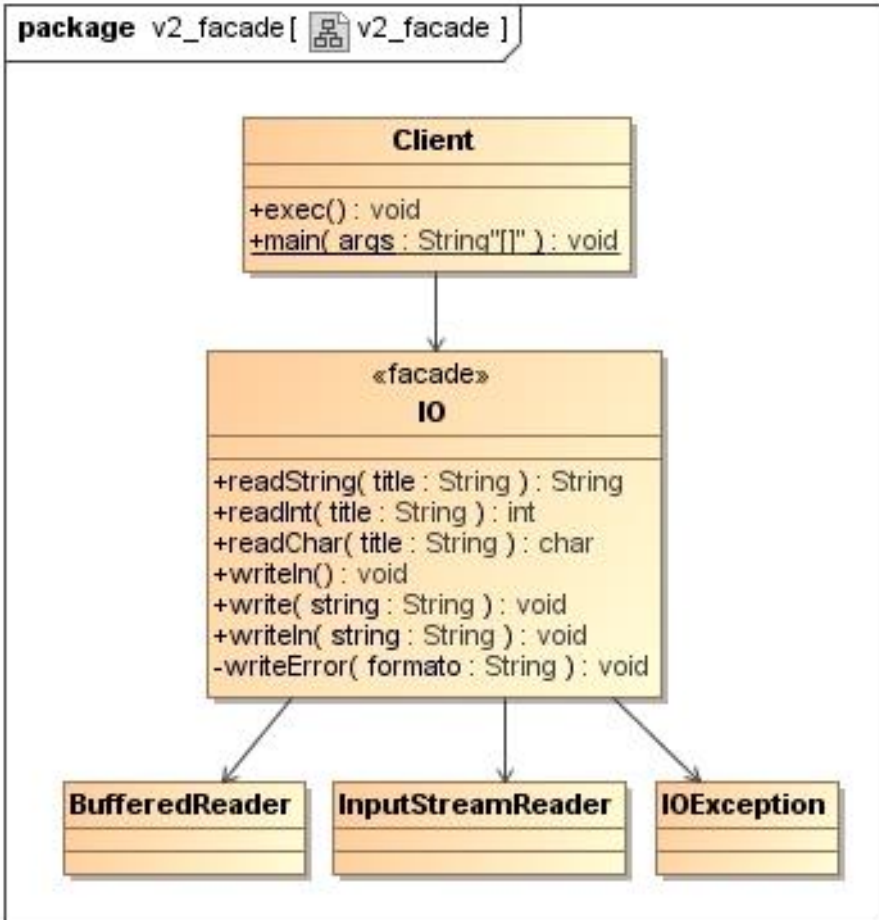
- Además, se prevé que en un futuro se opte por otra biblioteca (p.e. de *Apache*, ...)

# 1. Entrada/Salida en Modo Consola

- **Problema.** No se desea trabajar con un montón de objetos de distintas clases de java.io. Se desea un objeto de mayor entidad que encapsule a todos esos
  - Determinando la granularidad de los objetos: *Facade*
- **Intención.** Proveer de una interfaz unificada a un conjunto de interfaces de un subsistema definiendo una interfaz de más alto nivel que hace más fácil de usar el subsistema
- **Interrelación.** *Singleton*
- **Propósito.**
  - ~~*Facade vs Mediator.*~~ No se desea encapsular un interacción ...
  - ~~*Facade vs Abstract Factory.*~~ No se trata de facilitar una familia...

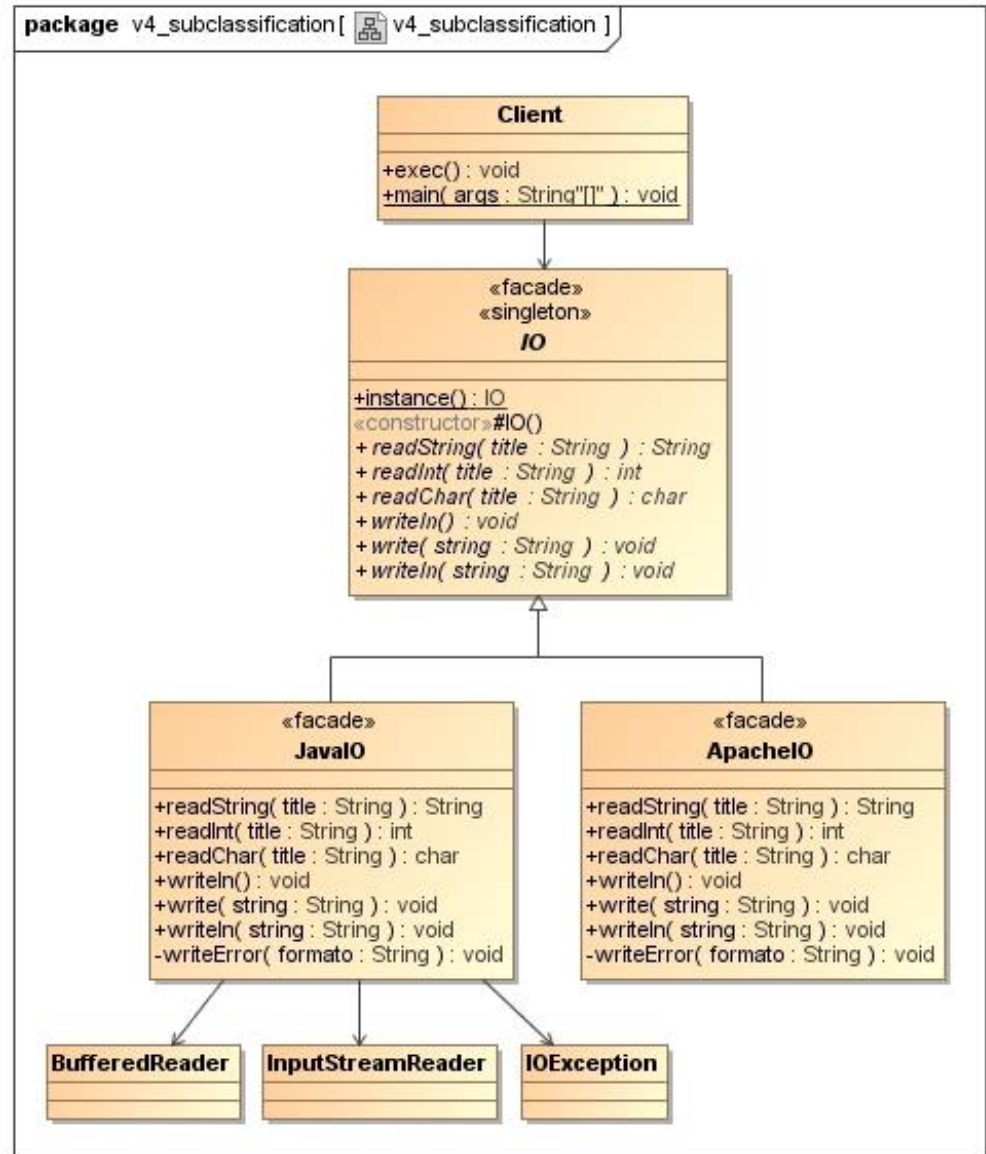


# 1. Entrada/Salida en Modo Consola



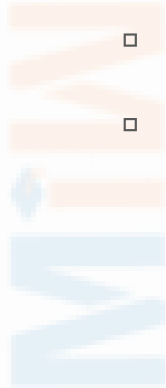
# 1. Entrada/Salida en Modo Consola

- **Rediseño.** Fuerte Acoplamiento a una fachada actual que deberá cambiar en el futuro
- **Variación.** La interfaz de un subsistema gracias a la sub-clasificación



# 1. Entrada/Salida en Modo Consola

- **Enunciado.** Se desean distintas funcionalidades asociadas a los métodos presentes en IO
  - Que toda letra se escriba en mayúsculas
  - Que toda letra sea subrayada
  - ...



# 1. Entrada/Salida en Modo Consola

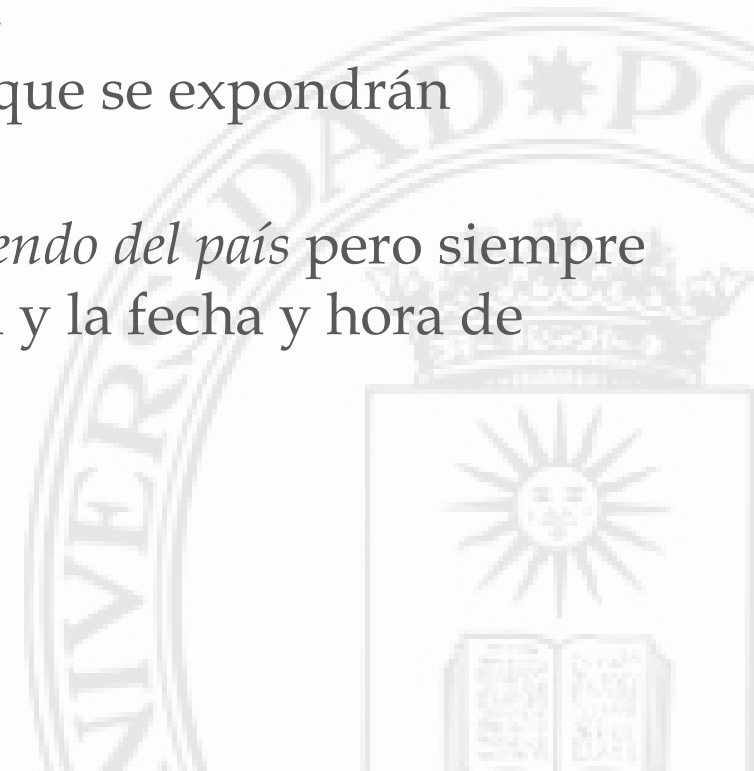
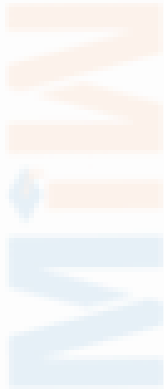
- **Problema.** No se desea complicar IO con cada nueva funcionalidad.
  - Relacionando estructuras de tiempo de compilación y ejecución: *Decorator*
- **Intención.** Agrega responsabilidades adicionales a un objeto dinámicamente, proveyendo una alternativa flexible a la subclasificación extendiendo la funcionalidad
- **Interrelación.** *Composite* y *Prototype*
- **Propósito.**
  - ~~*Decorator vs Adapter.*~~ No se trata de clases de bibliotecas, ...
  - ~~*Decorator vs Composite.*~~ No se trata abstraer 1 o N...
  - ~~*Decorator vs Strategy.*~~ Alternativa
  - ~~*Decorator vs Proxy.*~~ No se trata de sustitutos, ...

## 1. Entrada/Salida en Modo Consola

- **Rediseño.** Extendiendo funcionalidad sin sub-clasificación
- **Variación.** Las responsabilidades de un objeto sin subclasificación

## 2. Cadena Internacional de Supermercados

- **Enunciado.** Un supermercado internacional con cientos de centros repartidos por **23 países** desea una gestión de tickets.
  - Un **ticket** consta de:
    - **Cabecera:** con distintos campos *dependiendo del país* pero siempre el nombre el nombre de la cadena y la fecha y hora de comienzo del ticket de compra
    - **Cuerpo:** Una secuencia de líneas que se expondrán posteriormente
    - **Pie:** con distintos campos *dependiendo del país* pero siempre el nombre el nombre de la cadena y la fecha y hora de comienzo del ticket de compra



## 2. Cadena Internacional de Supermercados

- Una **línea** puede ser:
  - De **venta**, que refleja el código, descripción, unidades y precio
  - De **repetición**, que refleja otra línea exactamente igual a la dada por el número de la secuencia de líneas
  - De **anulación**, que refleja el número de línea anterior anulada con un saldo negativo igual a la línea que anula
  - De **devolución**, que refleja el código
  - ...
  - *Dependiendo del país* existirán unos tipos de líneas u otros aunque venta, repetición y anulación estás presentes en todos. Y, por ejemplo, en España existen devoluciones pero en Francia no.

## 2. Cadena Internacional de Supermercados

- Existen dos tipos de operaciones a realizar:
  - **Operación a nivel de línea** que se realiza con cada parte del ticket según se introduce en el sistema. Por ejemplo, calcular precio, visualizar, imprimir, descuentos por línea, ...
  - **Operación a nivel de ticket** que se realiza con todas las partes del ticket según se cierra. Por ejemplo, descuentos globales, precio global, modificación del almacén, ...
  - *Considerando el país*, se realizarán unas operaciones u otras. Por ejemplo, en España es legal mostrar la venta con un pequeño display monocromo y en Francia exigen una pantalla gráfica de 12'' mínimo; en España no hay descuentos globales y en Francia no hay descuentos por línea; ...



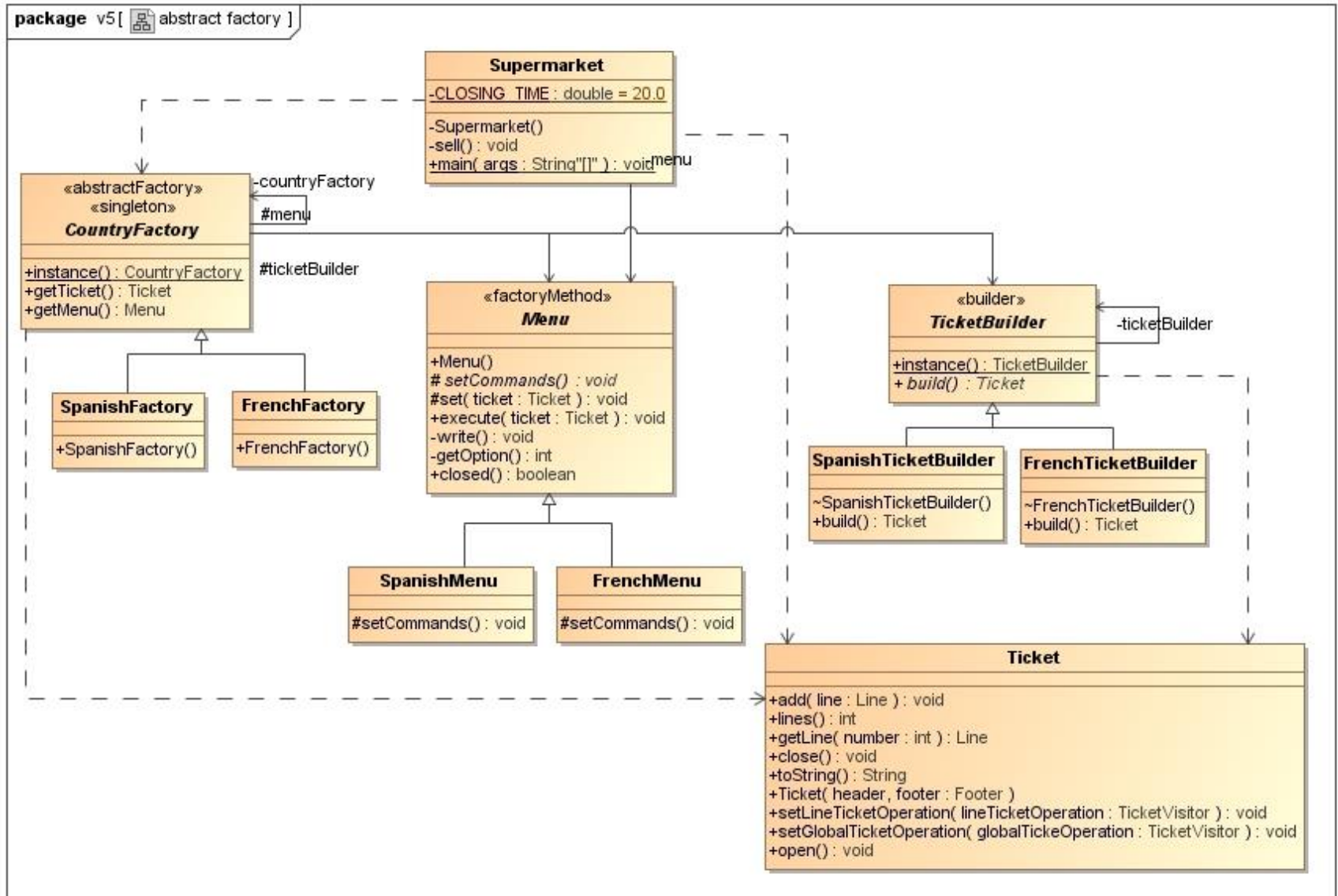
## 2. Cadena Internacional de Supermercados

- **Problema.** No se desea que *Supermarket* se complique con cada uno de los menús y tickets de cada país.
  - Programar para la interfaz, no para la implementación:
- **Abstract Factory**
- **Intención.** Provee una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas
- **Interrelación.** *Singleton, Factory Method y Prototype*
- **Propósito.**
  - ~~*Abstract Factory vs Facade.*~~ No se trata de ocultar un subsistema  
...
  - ~~*Abstract Factory vs Builder.*~~ No se trata de una creación compleja  
...

## 2. Cadena Internacional de Supermercados

- **Rediseño.** Creando un objeto especificando una clase explícitamente (*SpanishMenu* y *FrenchMenu*)
- **Variación.** Familias de objetos “producto” (derivadas de *Menu*)

## 2. Cadena Internacional de Supermercados



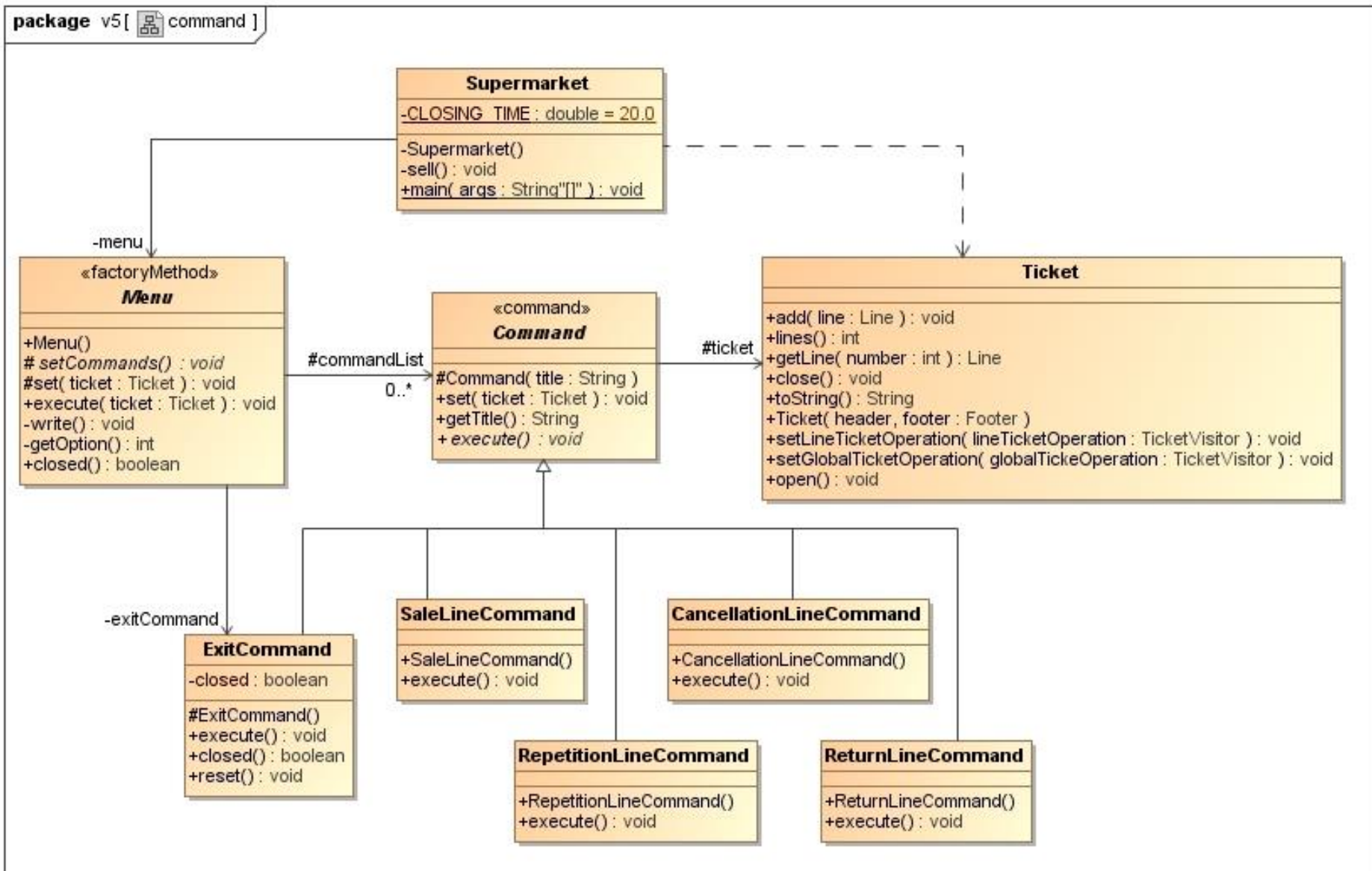
## 2. Cadena Internacional de Supermercados

- **Problema.** No se desea complicar los algoritmos del Menú con la particularidad de cada país.
  - Determinando la granularidad de los objetos : *Command*
- **Intención.** Encapsula un petición como un objeto, permitiendo de ese modo parametrizar clientes con diferentes peticiones, colas o solicitudes registradas, y apoyar las operaciones de deshacer.
- **Interrelación.** *Prototye*, ~~*Memento*~~ y ~~*Composite*~~
- **Propósito.**
  - ~~*Command vs Memento*~~. No se trata de recuperar estados anteriores ...

## 2. Cadena Internacional de Supermercados

- **Rediseño.** Dependencias de operaciones algorítmicas: podrá haber nuevas opciones
- **Variación.** Cómo y cuándo una operación es gestionada

## 2. Cadena Internacional de Supermercados

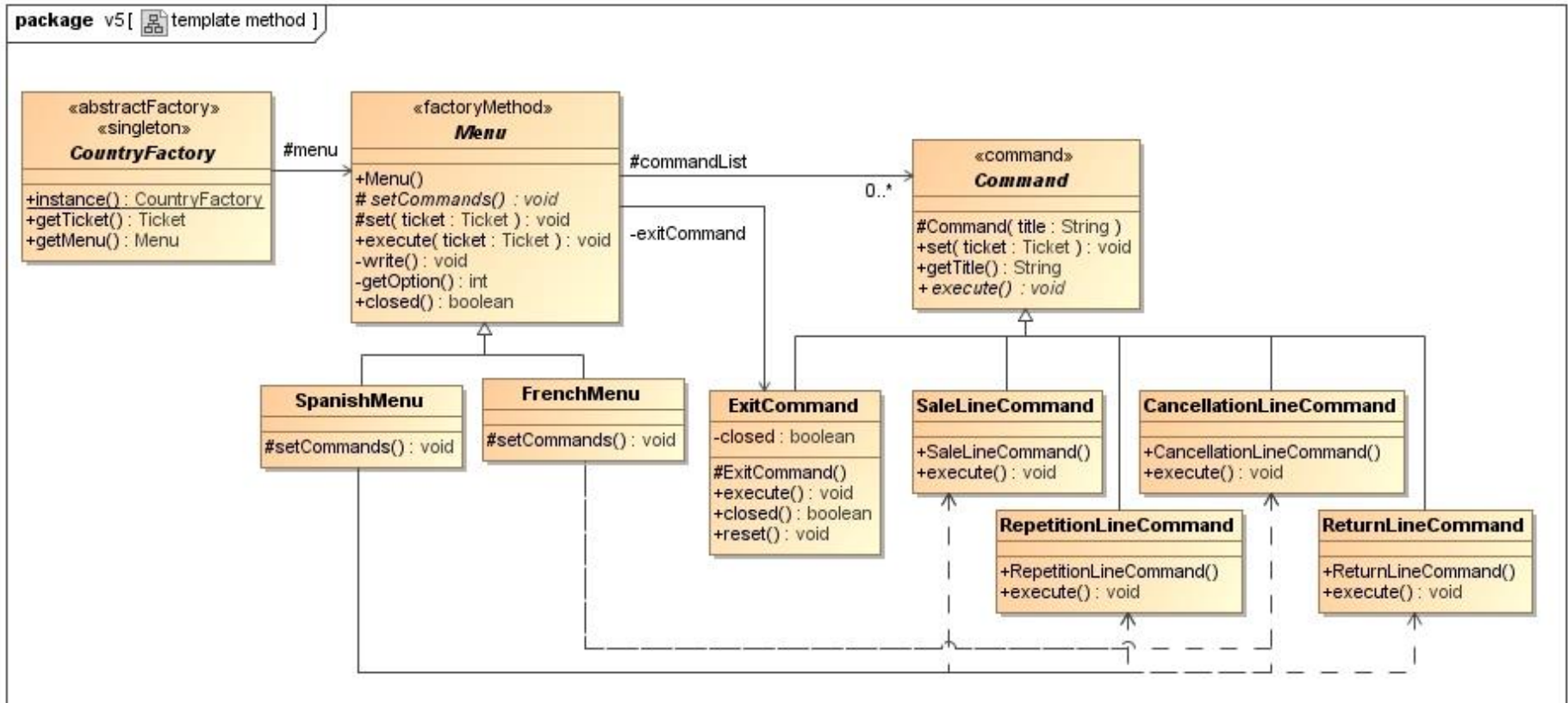


## 2. Cadena Internacional de Supermercados

- **Problema.** No se desea complicar los algoritmos del *Menú* con la particularidad de cada país.
  - Programando para la interfaz, no para la implementación:  
*Template Method*
- **Intención.** Define el esqueleto de un algoritmo en una operación difiriendo algunos pasos a las subclases que redefinen esos pasos del algoritmo sin cambiar la estructura
- **Interrelación.**
- **Propósito.**
  - ~~*Template Methos vs Strategy*~~. No se requiere dinamismo ...
- **Rediseño.** Dependencias algorítmicas de la configuración del menú por país
- **Variación.** Pasos de un algoritmo



## 2. Cadena Internacional de Supermercados





## 2. Cadena Internacional de Supermercados

- **Problema.** No se desea complicar los algoritmos del *Ticket* con la particularidad de cada país.
  - Determinando la granularidad de los objetos : *Visitor*
- **Intención.** Representa una operación para ser realizadas sobre los elementos de una estructura de objetos y permite definir nuevas operaciones sin cambiar las clases de elementos sobre los que opera
- **Interrelación.**
- **Propósito.**
- **Rediseño.** Inhabilidad para cambiar una clase adecuadamente: podrá haber nuevas operaciones
- **Variación.** Operaciones que pueden ser aplicadas a objetos sin cambiar su clase

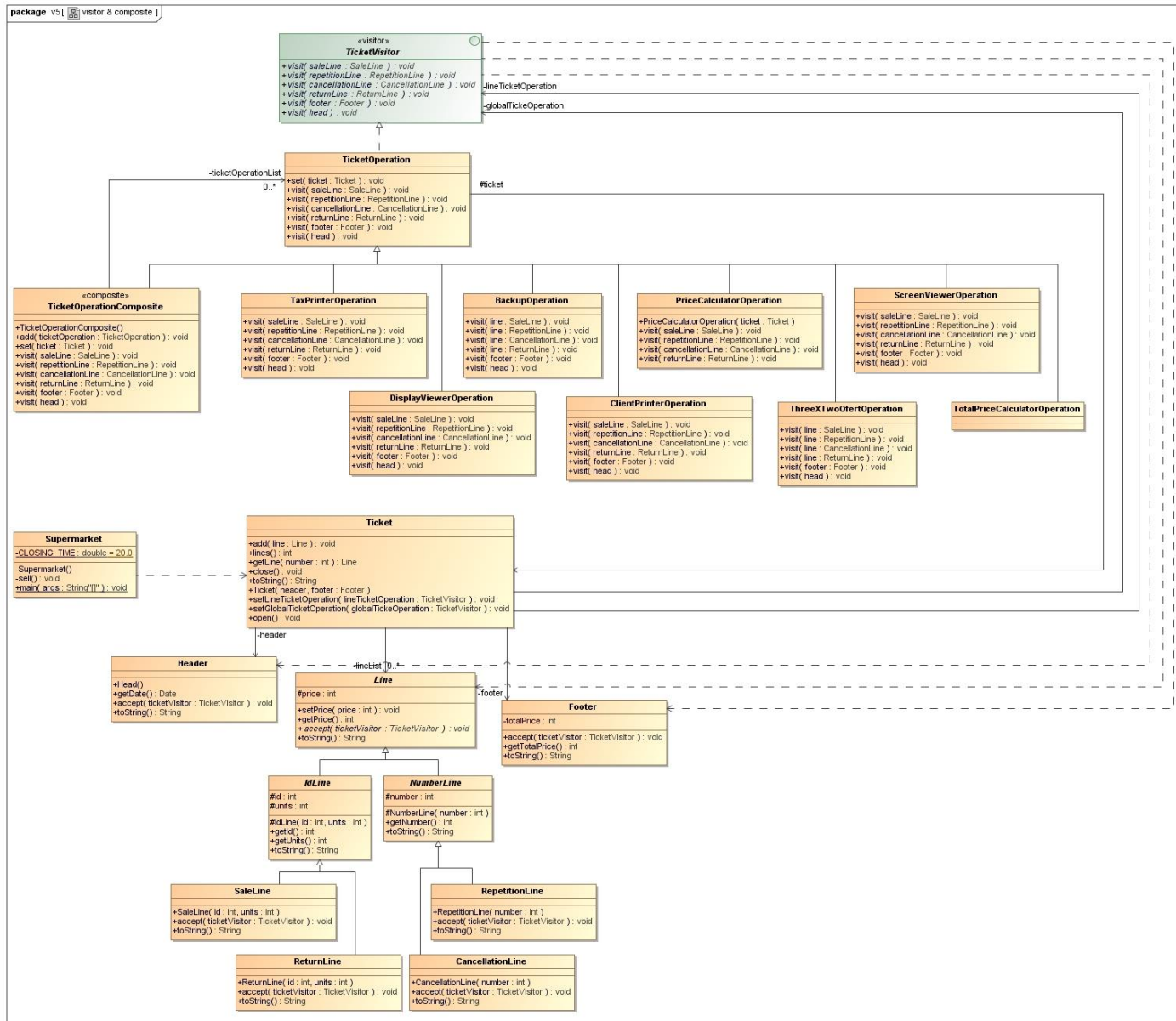
## 2. Cadena Internacional de Supermercados

- **Problema.** No se desea complicar al *Ticket* con la particularidad del número de operaciones.
  - Relacionando tiempo de compilación vs ejecución: *Composite*
- **Intención.** Compone objetos en estructuras arbóreas para representar la jerarquía de todo-parte y permite a los clientes tratar con objetos individuales y objetos compuestos uniformemente
- **Interrelación.** *Iterator, Visitor, Chain of Responsibility, ...*
- **Propósito.**
  - ~~*Composite vs Decorator*~~: No se pretende aportar funcionalidades dinámicamente ...
- **Rediseño.** Relacionando tiempo de compilación y de ejecución
- **Variación.** La estructura y composición de un objeto

## 2. Cadena Internacional de Supermercados

- **Problema.** No se desea complicar los algoritmos del *Ticket* con la particularidad de cada país.
  - Delegación: *Strategy*
- **Intención.** Define una familia de algoritmos, encapsula cada uno y los hace intercambiables. Permite al algoritmo variar independientemente del cliente que lo use
- **Interrelación.** *Flyweight*
- **Propósito.**
  - ~~*Strategy vs Decorator*~~. No interesa aportar nueva funcionalidad ...
  - ~~*Strategy vs Template Method*~~. Aumenta número de clases ...
- **Rediseño.** Dependencias algorítmicas
- **Variación.** Un algoritmo

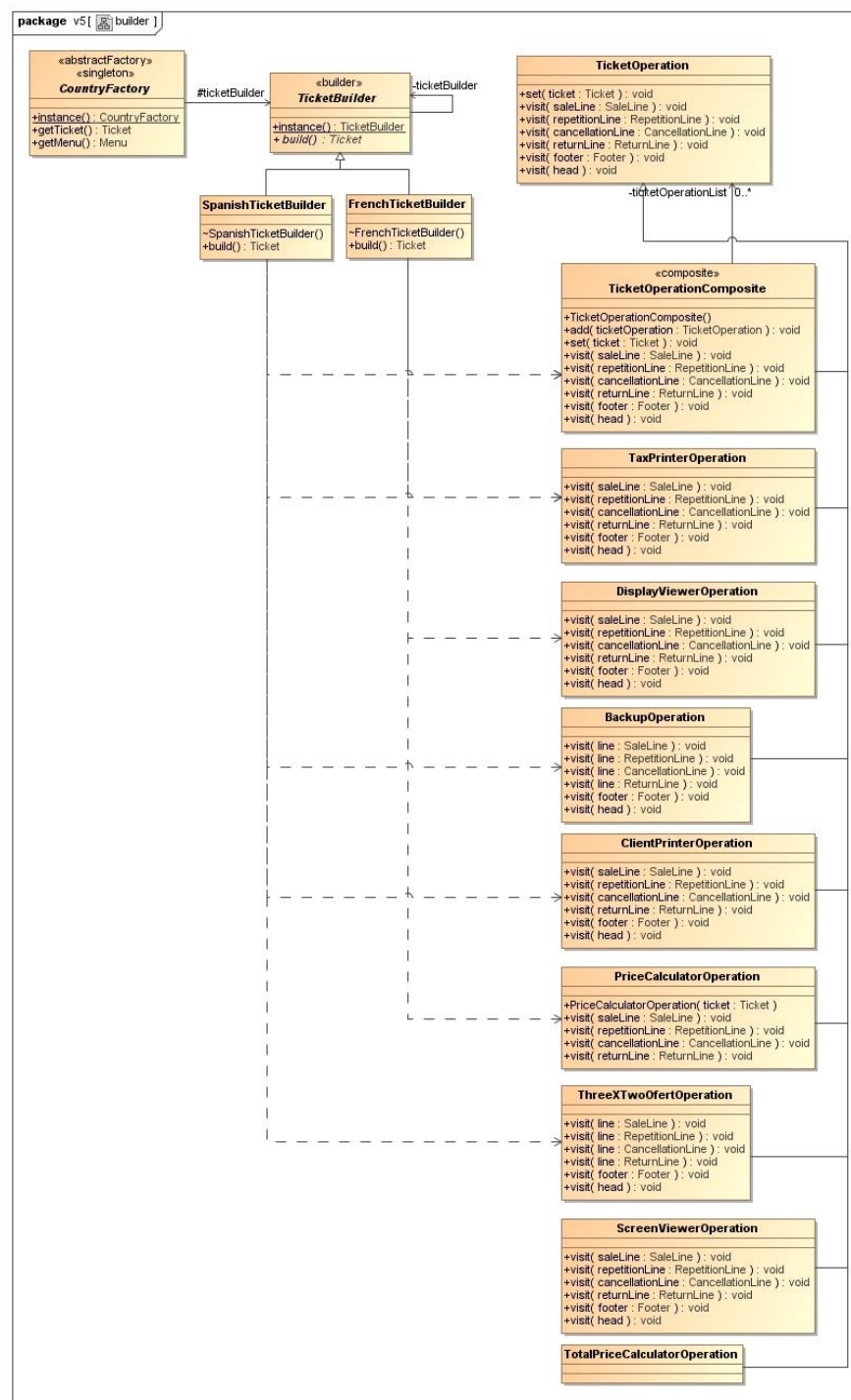
## 2. Cadena Internacional de Supermercados



## 2. Cadena Internacional de Supermercados

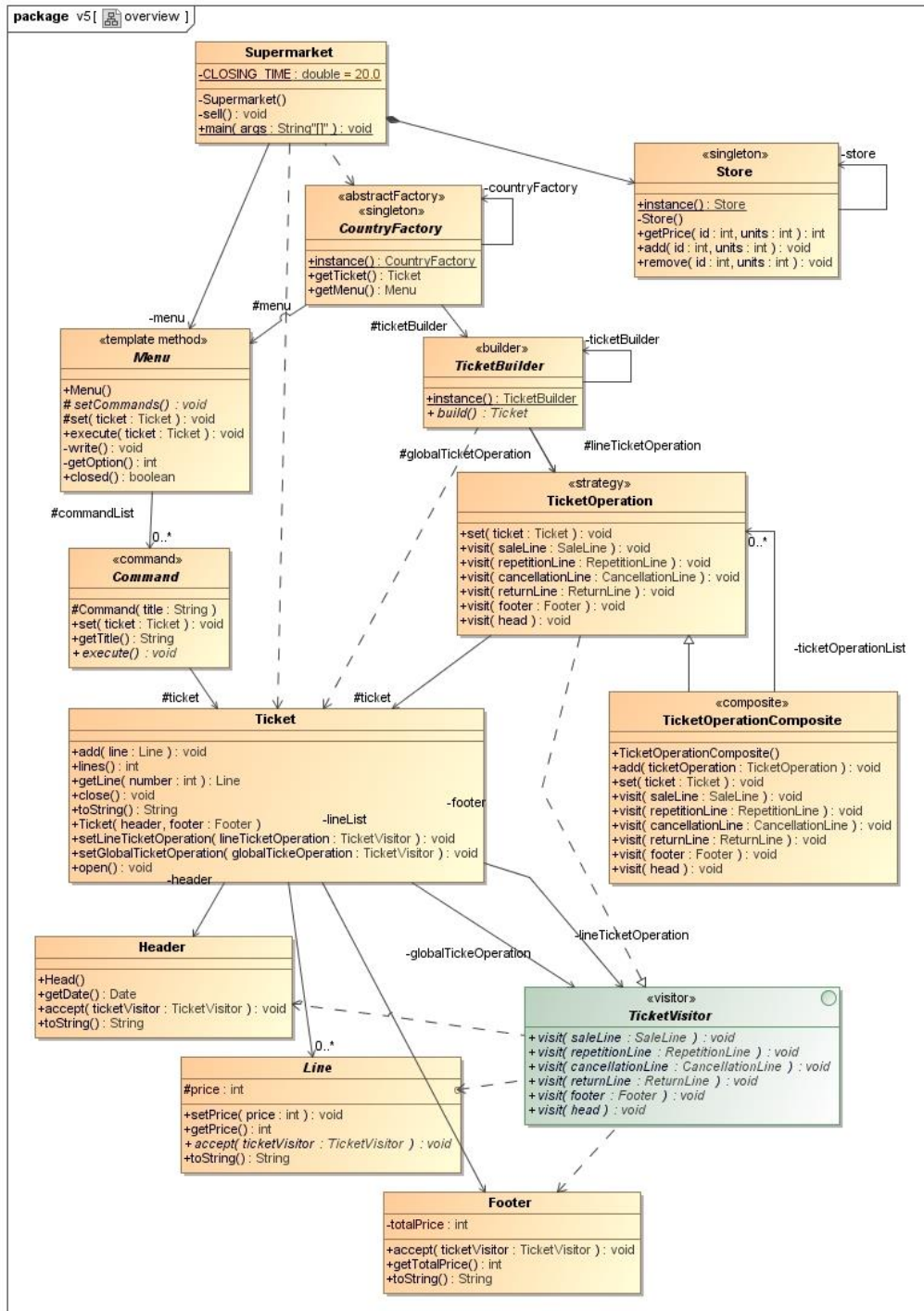
- **Problema.** No se desea complicar los algoritmos del *Ticket* con la particularidad de cada país.
  - Programando para la interfaz, no para la implementación:
- **Builder**
- **Intención.** Separa la construcción de un objeto complejo de su representación de tal forma que el mismo proceso de construcción puede crear diferentes representaciones
- **Interrelación.** *Singleton, Composite, Abstract Factory/Prototype*
- **Propósito.**
- **Rediseño.** Dependencias algorítmicas
- **Variación.** Cómo se crea un compuesto

## 2. Cadena Internacional de Supermercados



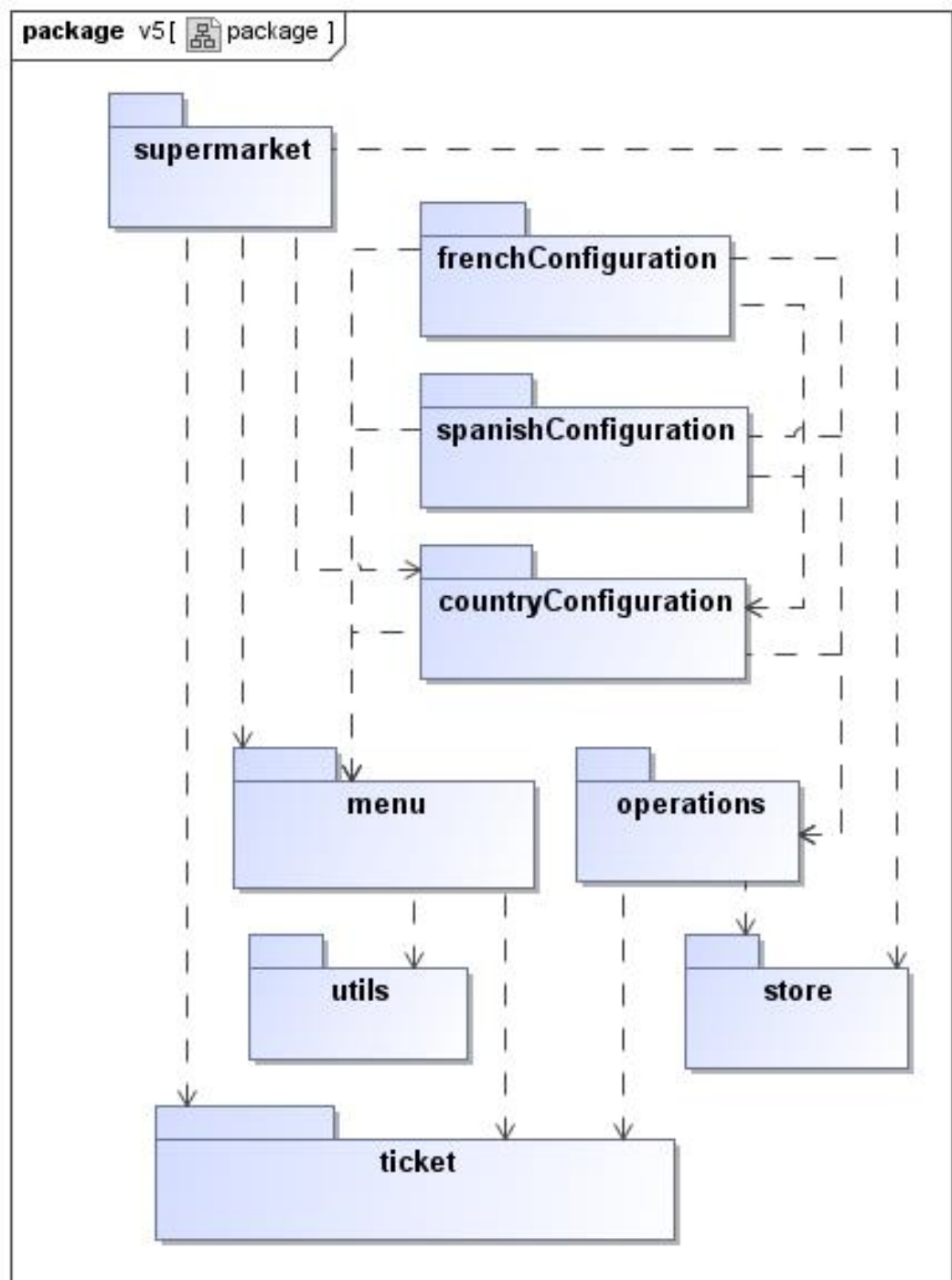


- **Visión general de clases principales**



## 2. Cadena Internacional de Supermercados

- Arquitectura del software (jerarquía de paquetes)





### 3. Gestor Local/Remoto OnLine/OffLine

- **Enunciado.** Se desea la gestión de una colección de productos identificados por su clave y su cantidad. Únicamente se contemplan operaciones de añadir y eliminar una cierta cantidad de un cierto código. Al inicio de la ejecución y tras cada operación se desea presentar el listado de códigos y cantidades vigentes. La colección de productos puede estar:
  - En local, sin comunicaciones
  - En remoto *on line*, con comunicaciones al servidor alojado en *localhost* y por el puerto 2020. Pero sin modificar toda la versión anterior, parte de la aplicación debe ser transparente al carácter local o remoto
  - En remoto *off line*, con *buffer* que almacena todas las peticiones para que el cliente siga siendo operativo y, cuando se recuperen las comunicaciones, se vuelca todo el *buffer* al servidor. Igualmente, el carácter *on line/off line* debe ser transparente para el resto de la aplicación

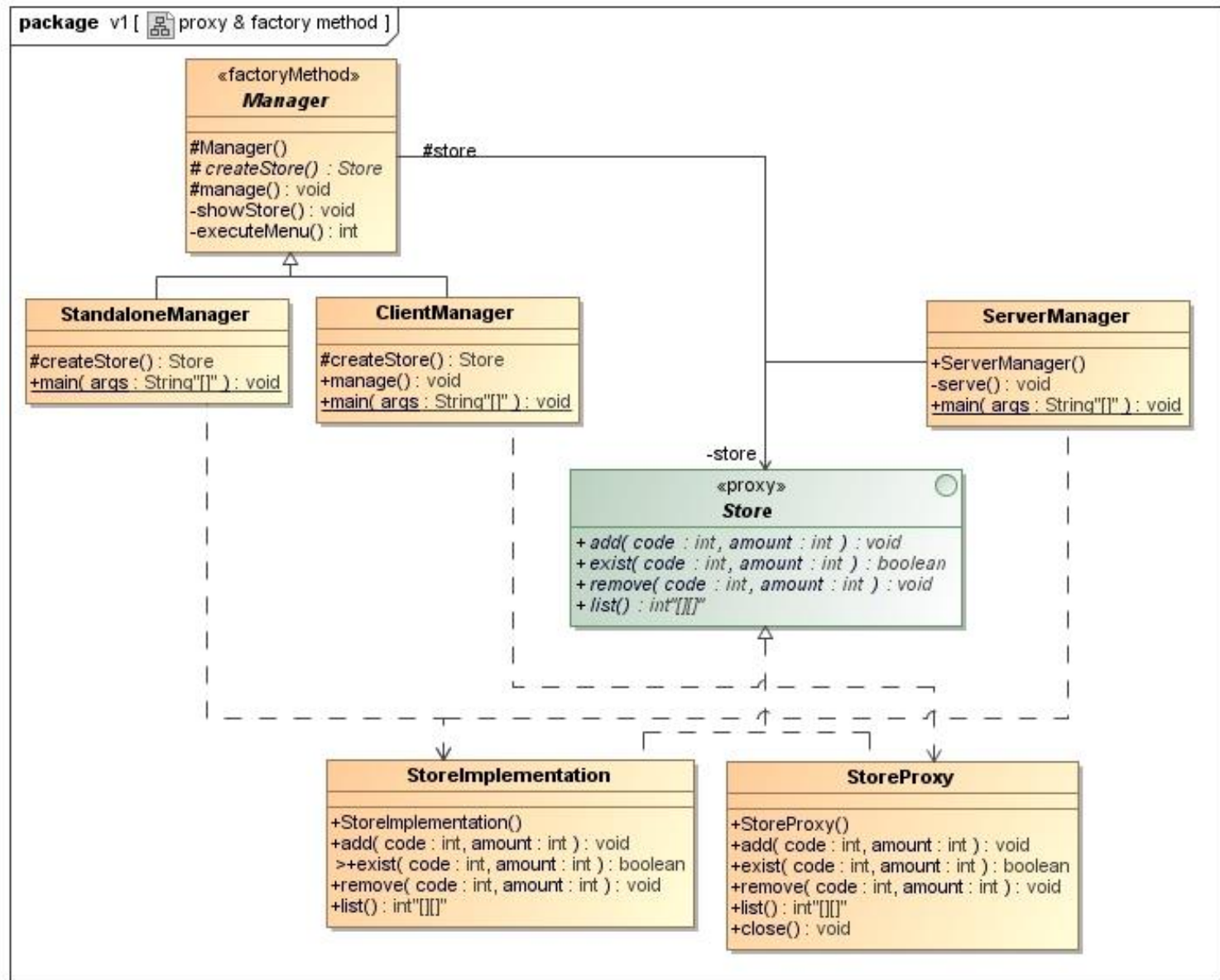
### 3. Gestor Local/Remoto OnLine/OffLine

- **Problema.** No se desea complicar al gestor con la particularidad del carácter local o remoto.
  - Especificando la interfaz de los objetos: *Proxy*
- **Intención.** Proporcionar un sustituto o representante de otro objeto para controlar el acceso al mismo.
- **Interrelación.**
- **Propósito.**
  - *Proxy vs Decorator*. No se trata de aumentar la funcionalidad
  - *Proxy vs Adapter*. No se trata de incorporar clases de biblioteca
- **Rediseño.** Dependencias de las representaciones o implementaciones de los objetos
- **Variación.** Cómo se accede a un objeto, su localización

### 3. Gestor Local/Remoto OnLine/OffLine

- **Problema.** No se desea complicar al gestor con la particularidad del carácter local o remoto.
  - Programando para la interfaz, no para la implementación:
- **Factory Method**
- **Intención.** Define una interfaz para crear un objeto pero permite a las subclases qué clase instanciar permitiendo a una clase diferir la instanciación a las subclases
- **Interrelación.**
- **Propósito.**
  - *Factory Method vs Template Method.* No se trata de definir un algoritmo ...
  - *Factory Method vs Prototype.* No se trata de crear copias ...
- **Rediseño.** Creando un objeto especificando una clase explíc.
- **Variación.** Subclase de objeto que es instanciado

### 3. Gestor Local/Remoto OnLine/OffLine



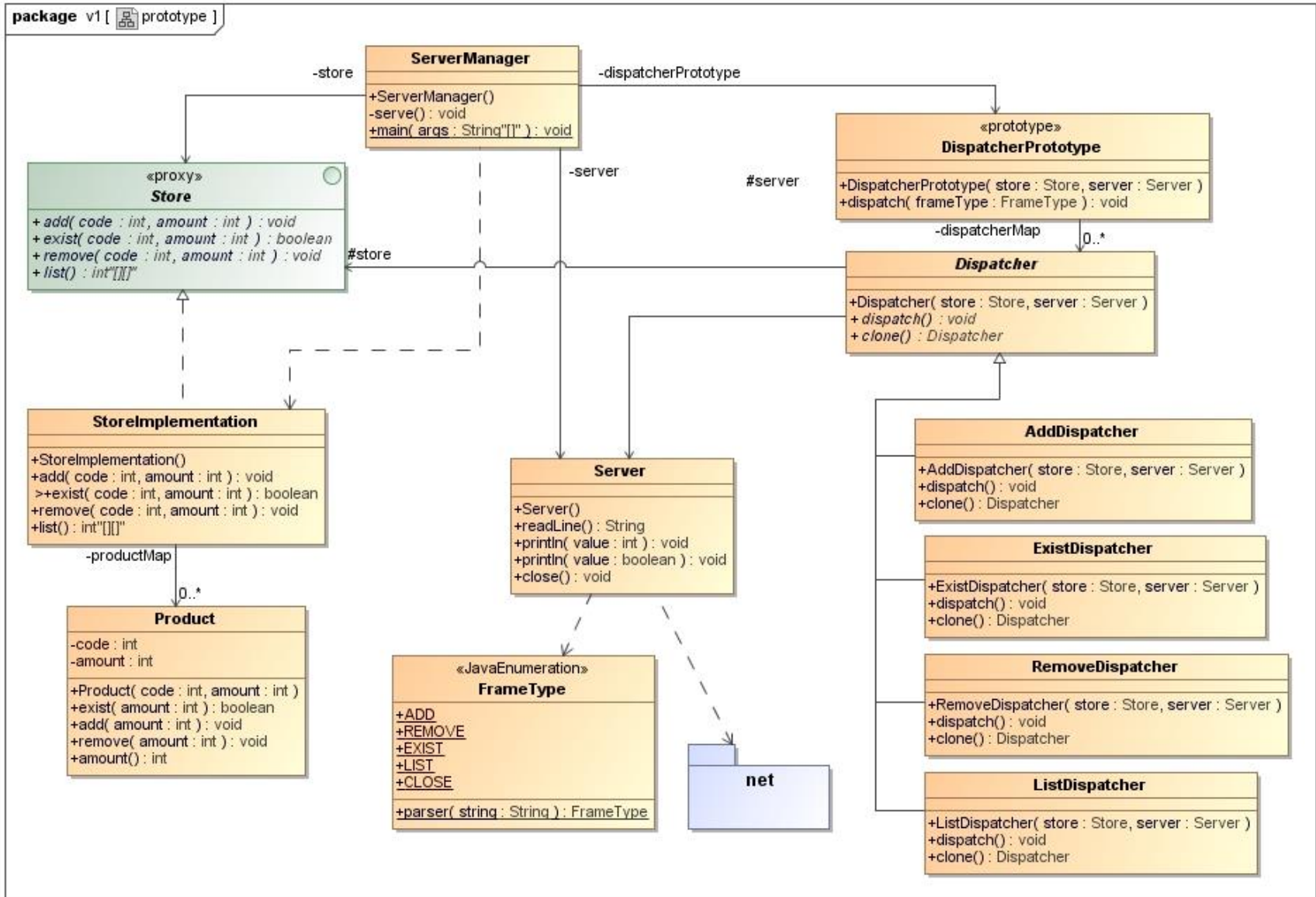
### 3. Gestor Local/Remoto OnLine/OffLine

- **Problema.** No se desea complicar al servidor con el análisis de cada trama que llegue.
  - Programando para la interfaz, no para la implementación:

#### *Prototype*

- **Intención.** Especifica las clases de objetos a crear usando una instancia prototípica y crea nuevos objetos por copias de este prototipo
- **Interrelación.** *Singleton*
- **Propósito.**
- **Rediseño.** Creando un objeto especificando una clase explíc.
- **Variación.** Clase de objeto que es instanciado

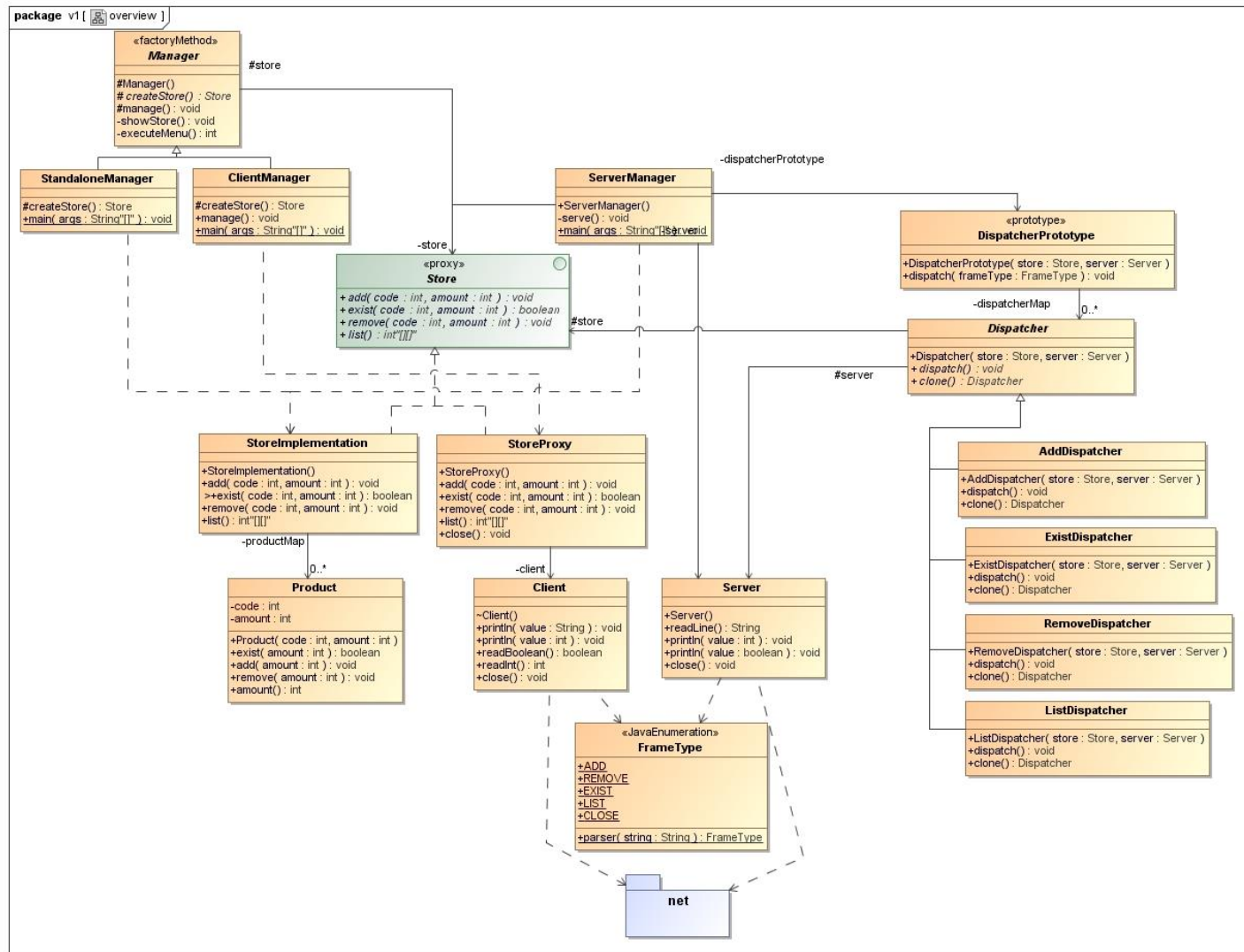
### 3. Gestor Local/Remoto OnLine/OffLine





# 3. Gestor Local/Remoto OnLine/OffLine

## ■ Visión general de clases principales



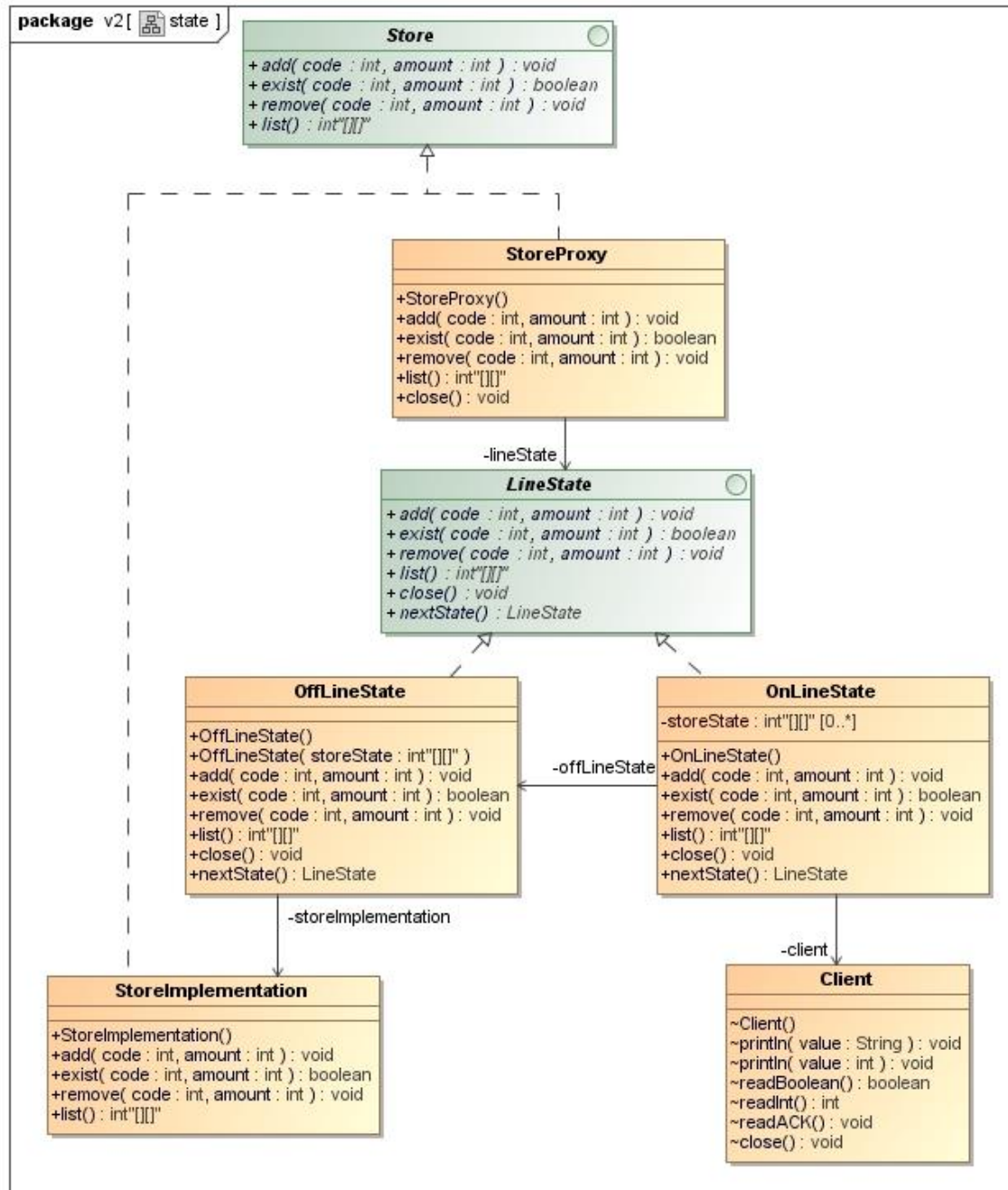


### 3. Gestor Local/Remoto OnLine/OffLine

- **Problema.** No se desea complicar al *StoreProxy* con la particularidad de si hay o no conexión.
  - Delegación: *State*
- **Intención.** Permite a un objeto alterar su comportamiento cuando su estado interno cambia pareciendo que el objeto cambia su clase
- **Interrelación.** *Singleton, Flyweight*
- **Propósito.**
- **Rediseño.**
- **Variación.** Estados de un objeto



# 3. Gestor Local/Remoto OnLine/OffLine



## 4. Juego del Poker

- **Enunciado.** Se desea desarrollar las bases para un juego de póker. En particular, determinar si un jugador tiene una jugada mejor, igual o peor que otro jugador. Las jugadas se listan de mejor a menor:
  - **Escalera Real**, cinco cartas consecutivas del 9 al Rey del mismo palo. Si ambas jugadas son escalera real, permanece el empate
  - **Escalera de Color**, cinco cartas consecutivas del mismo palo. Si ambas jugadas son escalera de color, gana la que tenga mayor el valor máximo de su escalera de color. En caso de empate, permanece el empate.
  - **Poquer**, cuatro cartas del mismo valor. Si ambas jugadas son poquer, gana la que tenga mayor el valor de las cuatro cartas del poquer. En caso de empate, gana la que tenga la otra carta más alta.

## 4. Juego del Poker

### ■ Enunciado.

- **Full**, tres cartas del mismo valor y las otras dos cartas del mismo valor. Si ambas jugadas son full, gana la que tenga mayor el valor de las tres cartas del full. En caso de empate, gana la que tenga mayor el valor de las dos cartas del full.
- **Color**, cinco cartas del mismo palo. Si ambas jugadas son color, gana la que tenga mayor la máxima carta de la jugada
- **Escalera**, cinco cartas consecutivas. Si ambas jugadas son escalera, gana la que tenga mayor el valor máximo de su escalera. En caso de empate, permanece el empate.
- **Trio**, tres cartas del mismo valor. Si ambas jugadas son trio, gana la que tenga mayor el valor de las tres cartas del poquer. En caso de empate, gana la que tenga la otra carta más alta. En caso de empate, gana la que tenga la última carta más alta. En caso de empate, permanece el empate.

## 4. Juego del Poker

### ■ Enunciado.

- **Doble pareja**, dos cartas del mismo valor y otras dos cartas del mismo valor. Si ambas jugadas son doble pareja, gana la que tenga mayor el valor de las dos parejas. En caso de empate, gana la que tenga mayor el valor de la otra pareja. En caso de empate, gana la que tenga mayor la última carta. En caso de empate, permanece el empate.
- **Pareja**, dos cartas del mismo valor. Si ambas jugadas son pareja, gana la que tenga mayor el valor de las dos cartas del poquer. En caso de empate, gana la que tenga la otra carta más alta. En caso de empate, gana la que tenga la siguiente carta más alta. En caso de empate, gana la que tenga mayor la última carta. En caso de empate, permanece el empate.
- **Carta alta**. Si ambas jugadas son carta alta, gana la que tenga mayor la carta mayor de la jugada. En caso de empate, la siguiente. En caso de empate, ...

## 4. Juego del Poker

- **Problema.** No se desea complicar al jugador con todas las condiciones para determinar un vencedor.
  - Relacionando estructuras en tiempo de compilación y de ejecución: *Chain of Responsibility*
- **Intención.** Evita acoplar al emisor de una petición a su receptor por dar más de un objeto una oportunidad de manejar la petición. Encadenar los objetos que reciben y pasar la solicitud a lo largo de la cadena hasta un objeto que la maneje.
- **Interrelación.** *Template Method, Observer, Command, Composite, State*
- **Propósito.**
- **Rediseño.** Dependencia de operaciones específicas
- **Variación.** El objeto que puede completar una petición

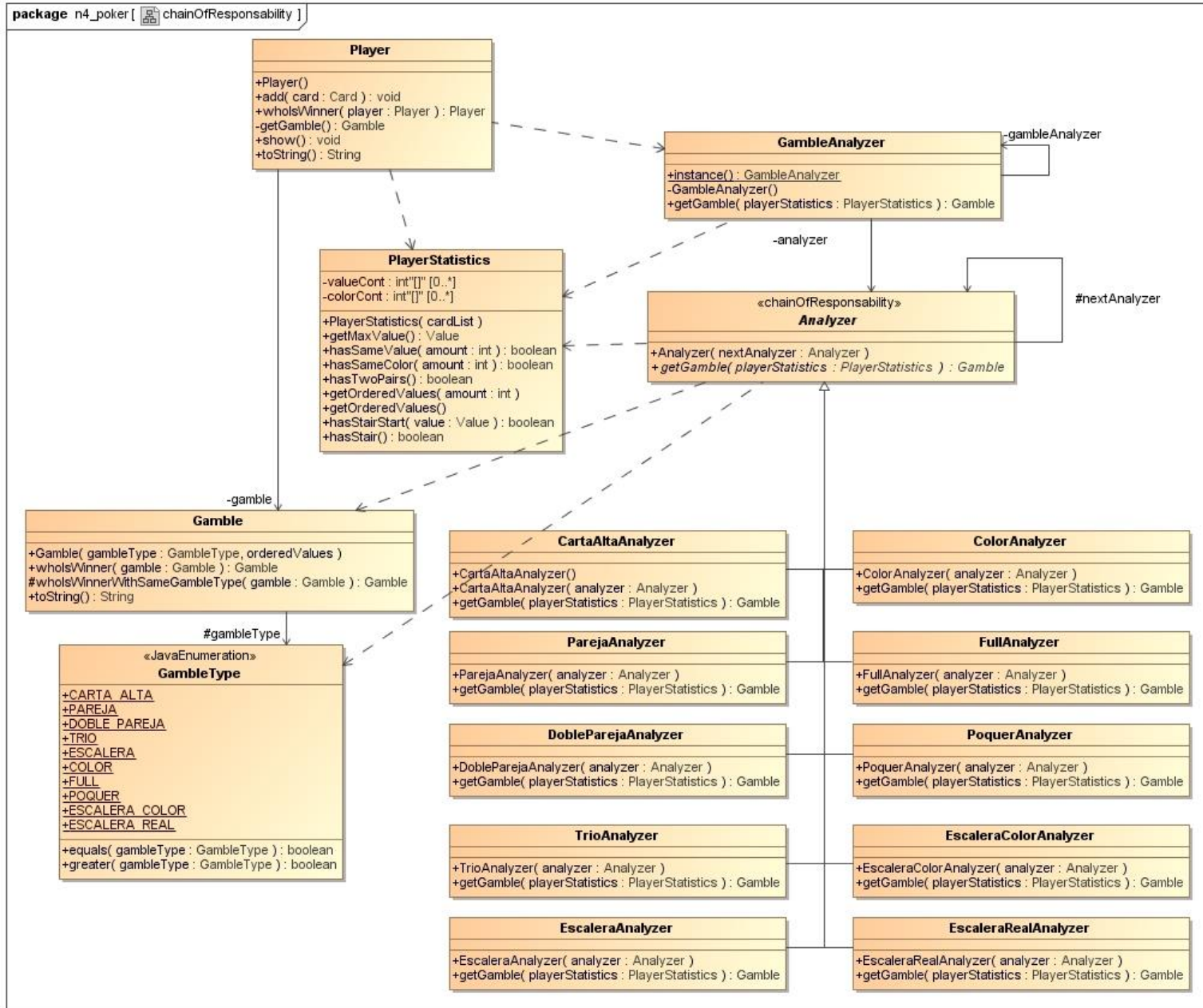
## 4. Juego del Poker

- **Problema.** No se desea crear distintas cartas por cada baraja que se crea.
  - Determinando la granularidad de los objetos: *Flyweight*
- **Intención.** Uso compartido para soportar gran número de objetos de grano fino eficientemente
- **Interrelación.**
- **Propósito.**
- **Rediseño.**
- **Variación.** Coste de almacenamiento de los objetos

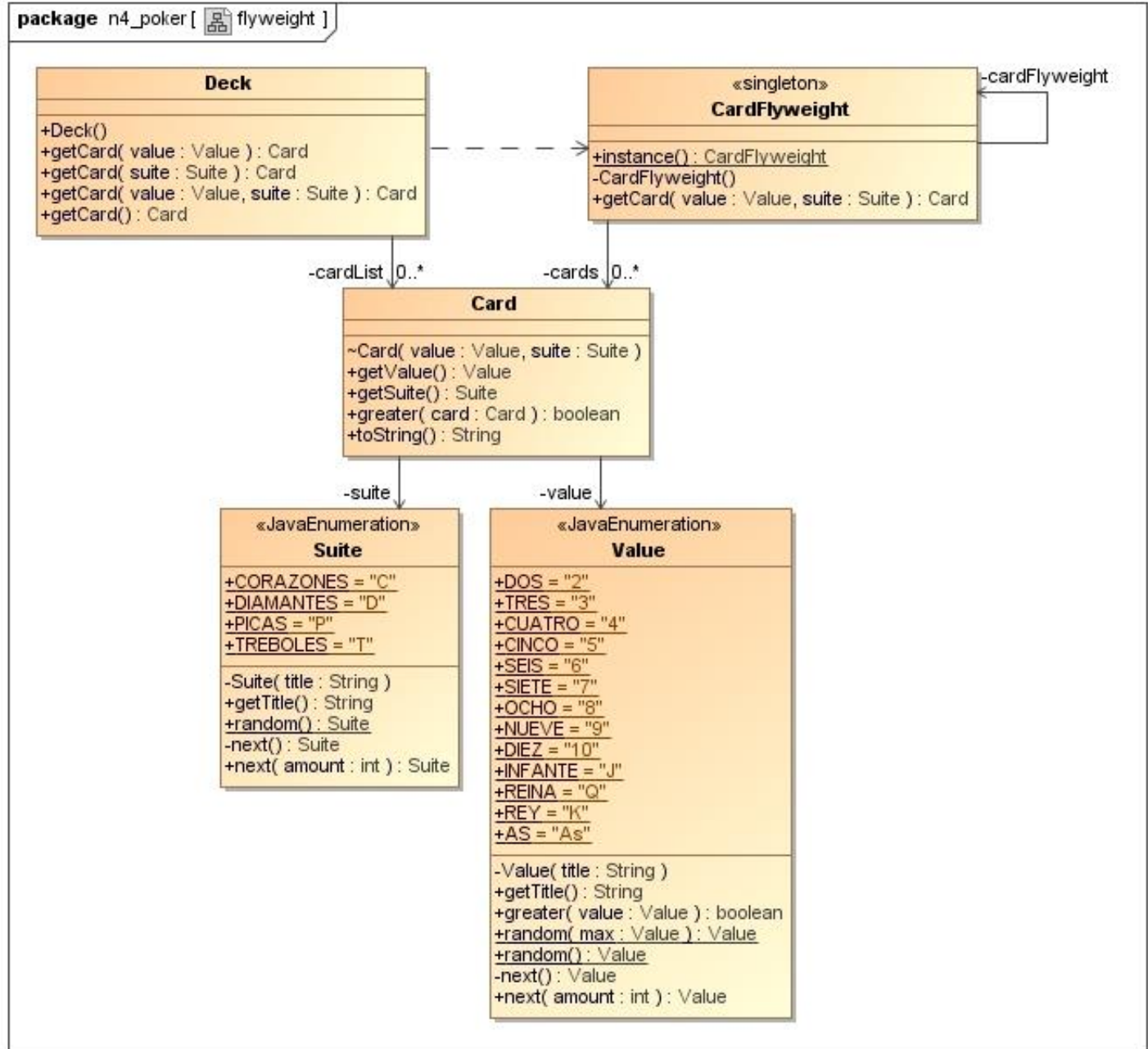


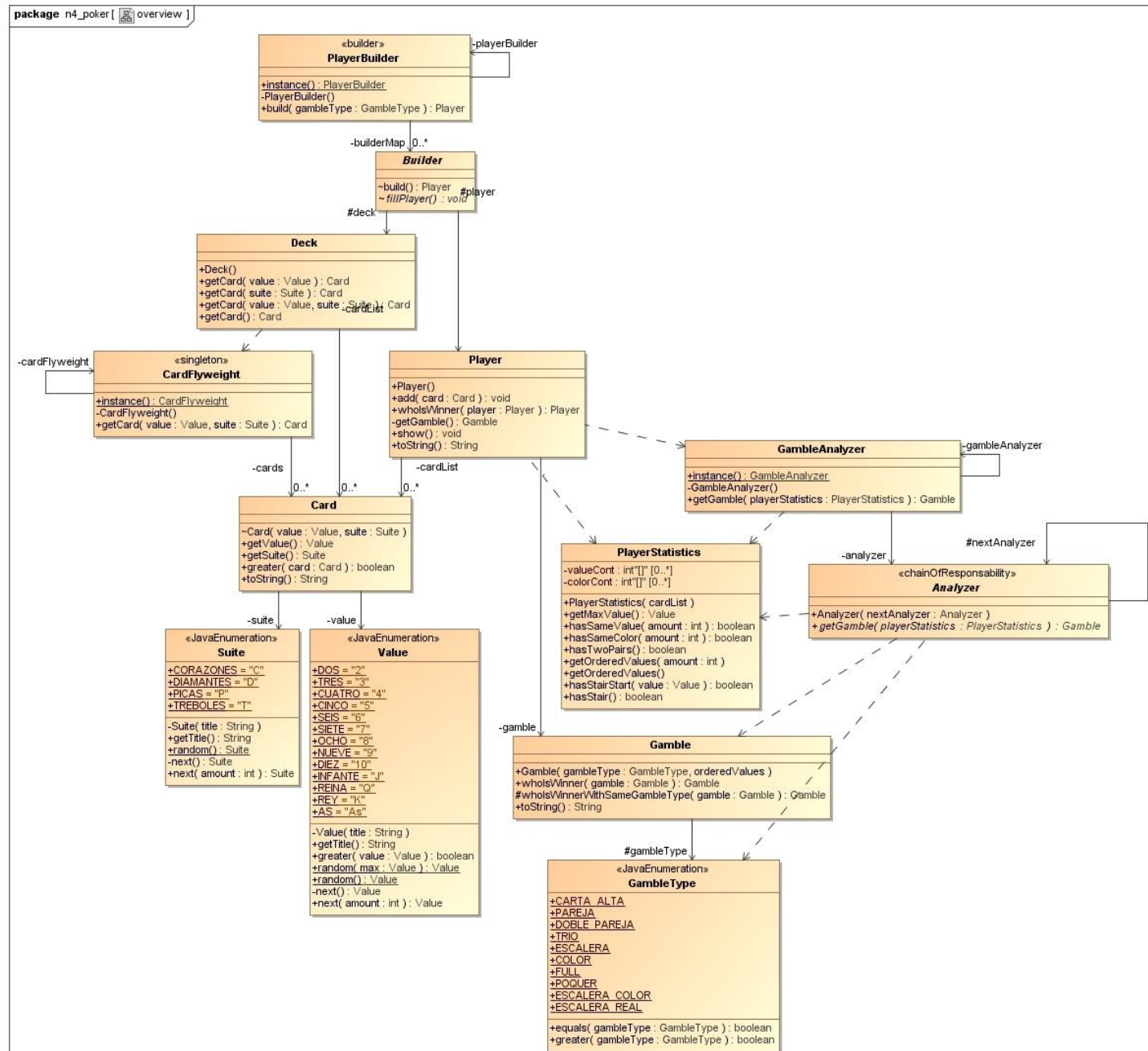


# 4. Juego del Poker

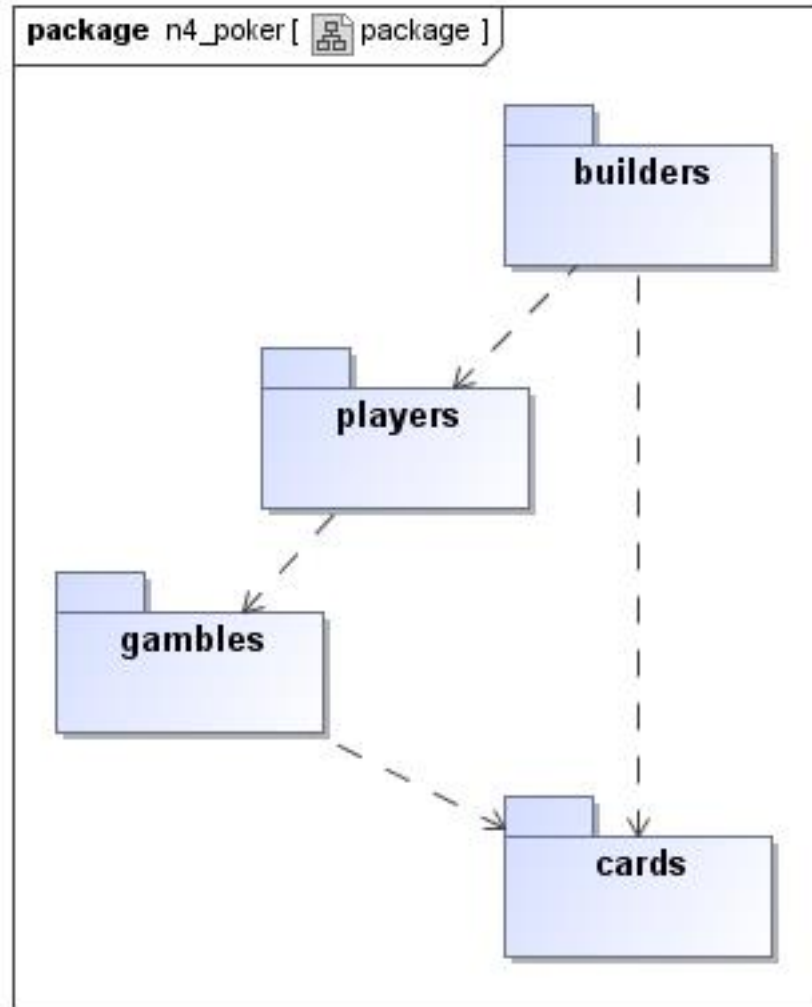


## 4. Juego del Poker





## 4. Juego del Poker



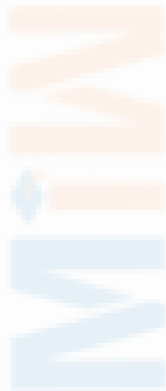
## 5. Máquina de Venta de Café

- **Enunciado.** Se desarrollar un prototipo del software empotrado de una máquina de venta de café. La interacción con el usuario será a través de consola, evitando la interfaz con todos los sensores.
  - La máquina dispone de un tanque de agua, un depósito de azúcar y un cargador de cartuchos de café,
  - La máquina ofrece una serie de productos (p.e. café solo, café con leche, café americano, ...). Cada producto tiene una descripción y tiene un cierto consumo de agua, azúcar y café.
  - El menú del prototipo tiene opciones para la selección de cada producto y opciones de gestión.
    - Tras la selección de cualquier producto se presenta un submenú para la selección del azúcar: nada; poco; medio; mucho; muchísimo. Tras la venta de cualquier producto pueden desactivarse todas las opciones por ausencia de la cantidad mínima necesaria en el tanque, depósito y cargador.

## 5. Máquina de Venta de Café

### ■ Enunciado.

- Tras la selección de gestión se presentará un submenú con opciones para rellenar el tanque, depósito y el cargador, con unidades de mililitros, miligramos y unidades respectivamente. Tras la ejecución de cualquier opción pueden reactivarse todas las opciones del menú de venta por la superación de la cantidad mínima necesaria en el tanque, depósito y/o cargador.





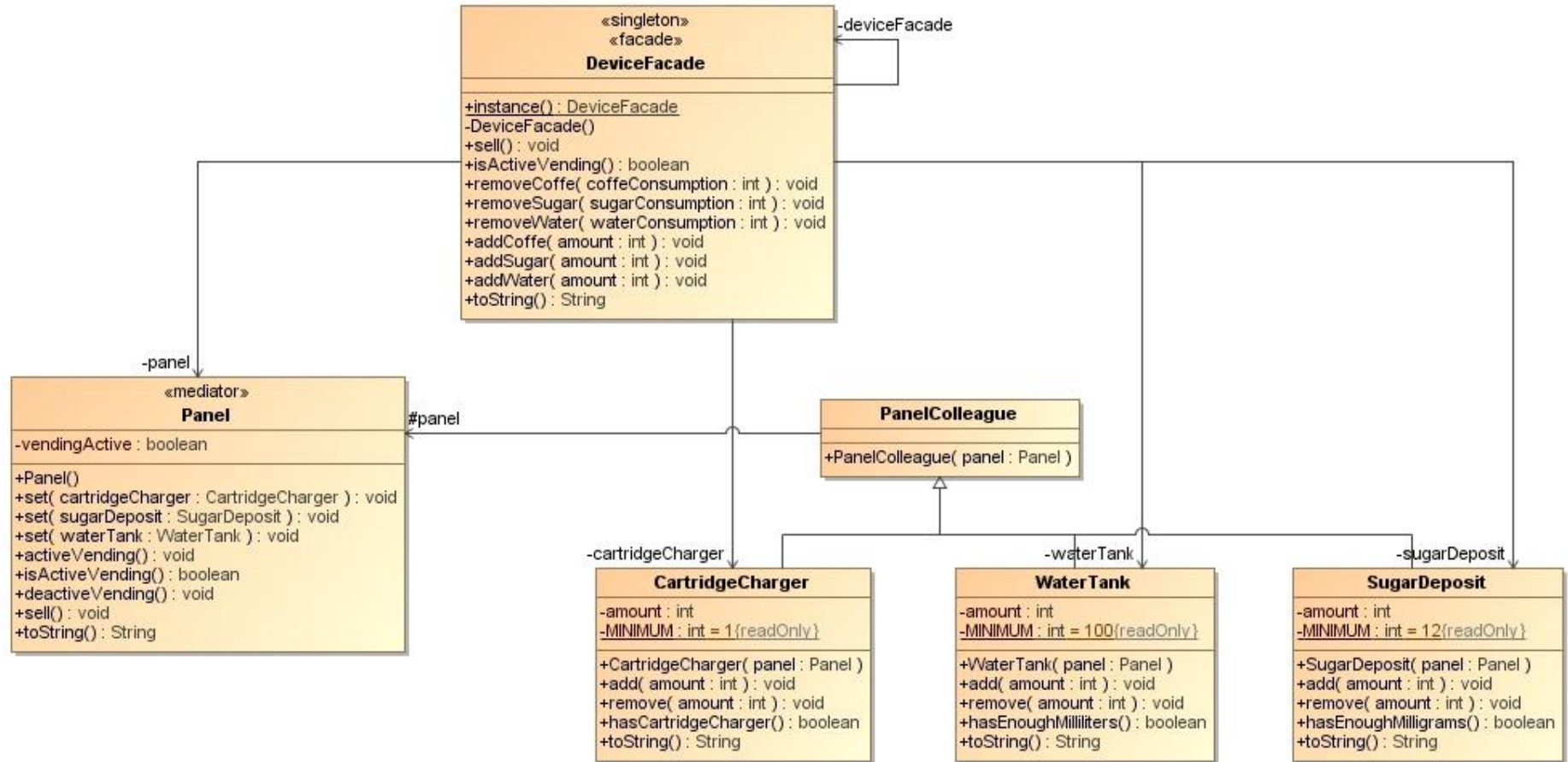
## 5. Máquina de Venta de Café

- **Problema.** No se desea que todo dispositivo conozca a todos los demás dispositivos.
  - Delegación: *Mediator*
- **Intención.** Un objeto que encapsula cómo interactúan un conjunto de objetos. Promueve bajo acoplamiento evitando a los objetos referirse explícitamente a otros y permite variar su interacción de forma independiente
- **Interrelación.** *Observer*
- **Propósito.**
  - *Mediator vs Facade.* No se trata de ocultar un subsistema
- **Rediseño.** Fuerte acoplamiento
- **Variación.** Cómo y qué objetos colaboran con otros

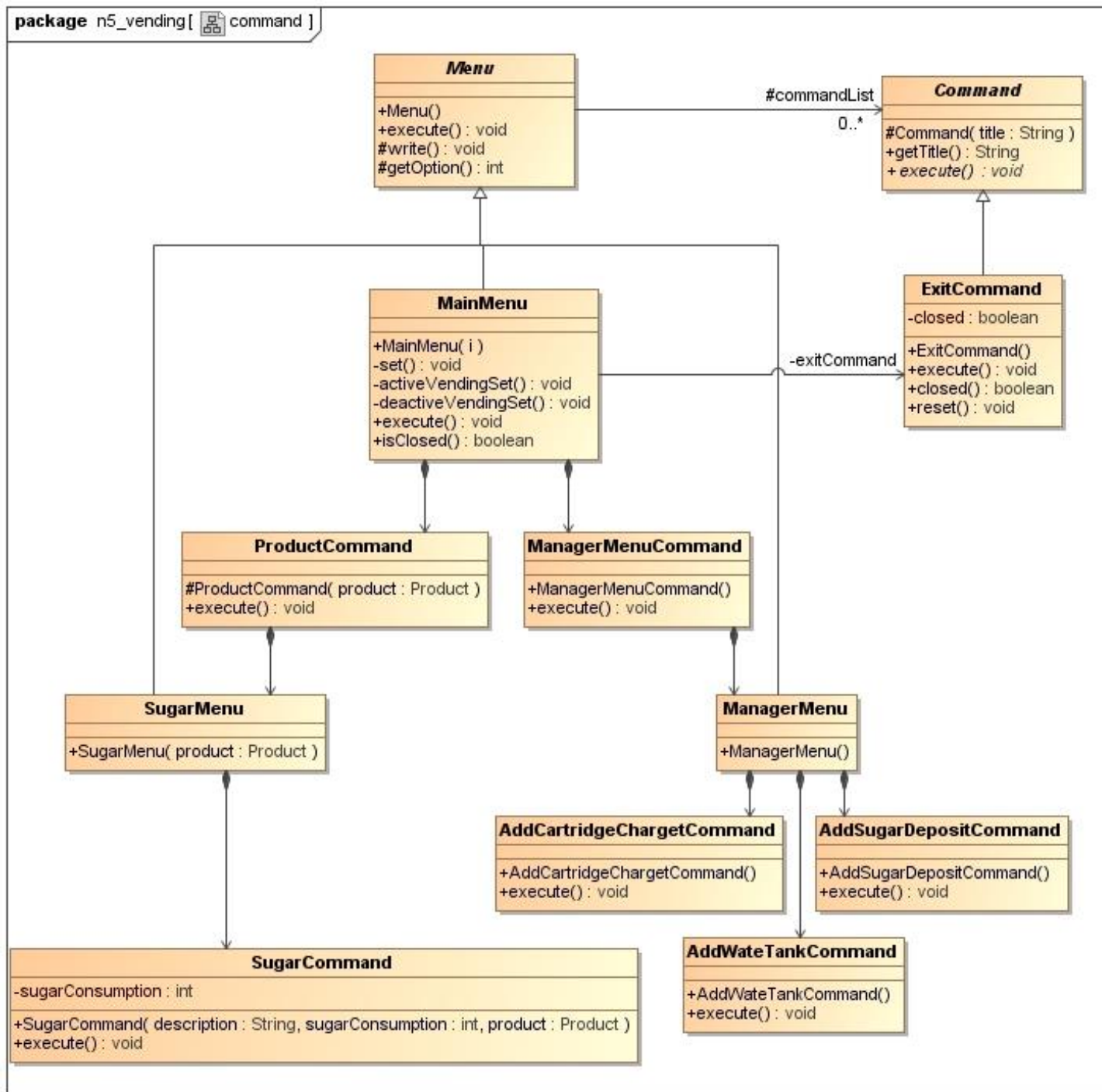


## 5. Máquina de Venta de Café

package n5\_vending [  mediator ]



# 5. Máquina de Venta de Café



## 5. Máquina de Venta de Café

- **Problema.** No se desea que todo dispositivo conozca a todos los demás dispositivos.
  - Delegación: *Observer*
- **Intención.** Un objeto que encapsula cómo interactúan un conjunto de objetos. Promueve bajo acoplamiento evitando a los objetos referirse explícitamente a otros y permite variar su interacción de forma independiente
- **Interrelación.** *Observer*
- **Propósito.**
  - *Mediator vs Facade.* No se trata de ocultar un subsistema
- **Rediseño.** Fuerte acoplamiento
- **Variación.** Cómo y qué objetos colaboran con otros

## 6. Juego de las Tres En Raya

- **Enunciado.** Se desea un prototipo del juego de las tres en raya mediante un menú dinámico con las siguientes opciones:
  - Poner ficha.
    - *Activación:* cuando no existen 6 fichas en el tablero.
    - *Acción:* solicitar coordenada de destino hasta que sea correcta (con valores de fila y columna entre 1 y 3 y que no esté ocupada). Registrar la ficha en el tablero para el jugador ('x' ó 'o') del turno actual y cambiar el turno.
  - Mover ficha.
    - *Activación:* cuando existen 6 fichas en el tablero.
    - *Acción:* solicitar coordenada de origen y destino hasta que sea correcta (con valores de fila y columna entre 1 y 3 y que esté ocupada el origen por ficha del jugador actual y vacía el destino). Eliminar la ficha origen y registrar la ficha destino en el tablero para el jugador ('x' ó 'o') del turno actual y cambiar el turno.

## 6. Juego de las Tres En Raya

### ■ Enunciado.

#### ▫ Deschacer ficha.

- *Activación*: cuando existe la ejecución previa de alguna opción de poner o mover.
- *Acción*: presentar el juego en el estado anterior a la ejecución previa de la última opción de poner o mover.

#### ▫ Rehacher ficha.

- *Activación*: cuando existe la ejecución previa de alguna opción de poner o mover deshecha.
- *Acción*: presentar el juego en el estado posterior a la ejecución previa de la última opción de poner o mover deshecha.

#### ▫ Rehacer juego.

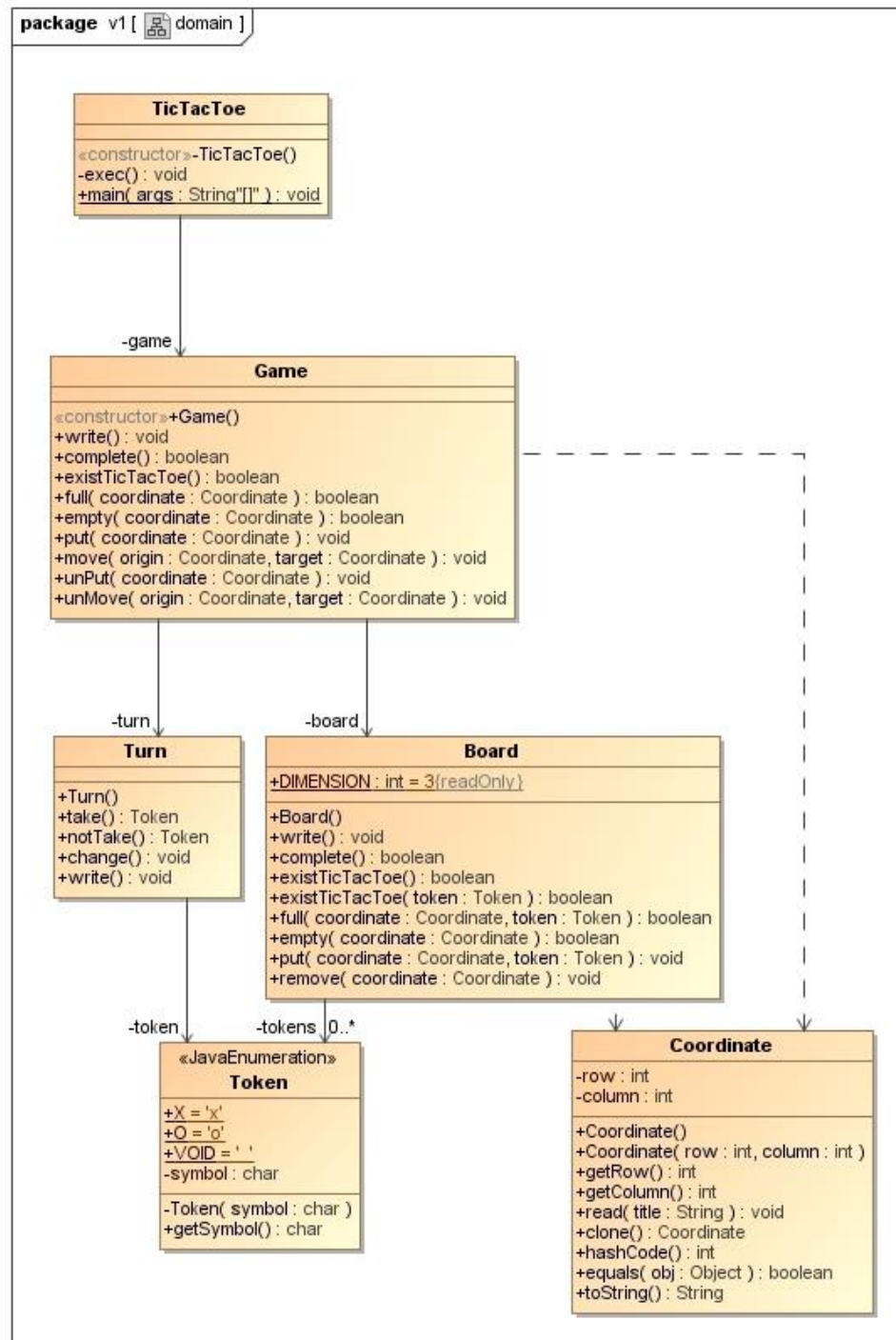
- *Activación*: siempre.
- *Acción*: sale del bucle de ejecución del menú al que pertenece.

## 6. Juego de las Tres En Raya

- **Problema.** Se desea gestionar acciones dependiendo de ciertas condiciones.
  - Determinando la granularidad de los objetos: *Command*
- **Intención.** Encapsula un petición como un objeto, permitiendo de ese modo parametrizar clientes con diferentes peticiones, colas o solicitudes registradas, y apoyar las operaciones de deshacer
- **Interrelación.** *Prototype, Composite, Memento*
- **Propósito.**
  - *Composite vs Memento.* Combinables
- **Rediseño.** Dependencias de operaciones específicas
- **Variación.** Cómo y cuándo las operaciones son completadas

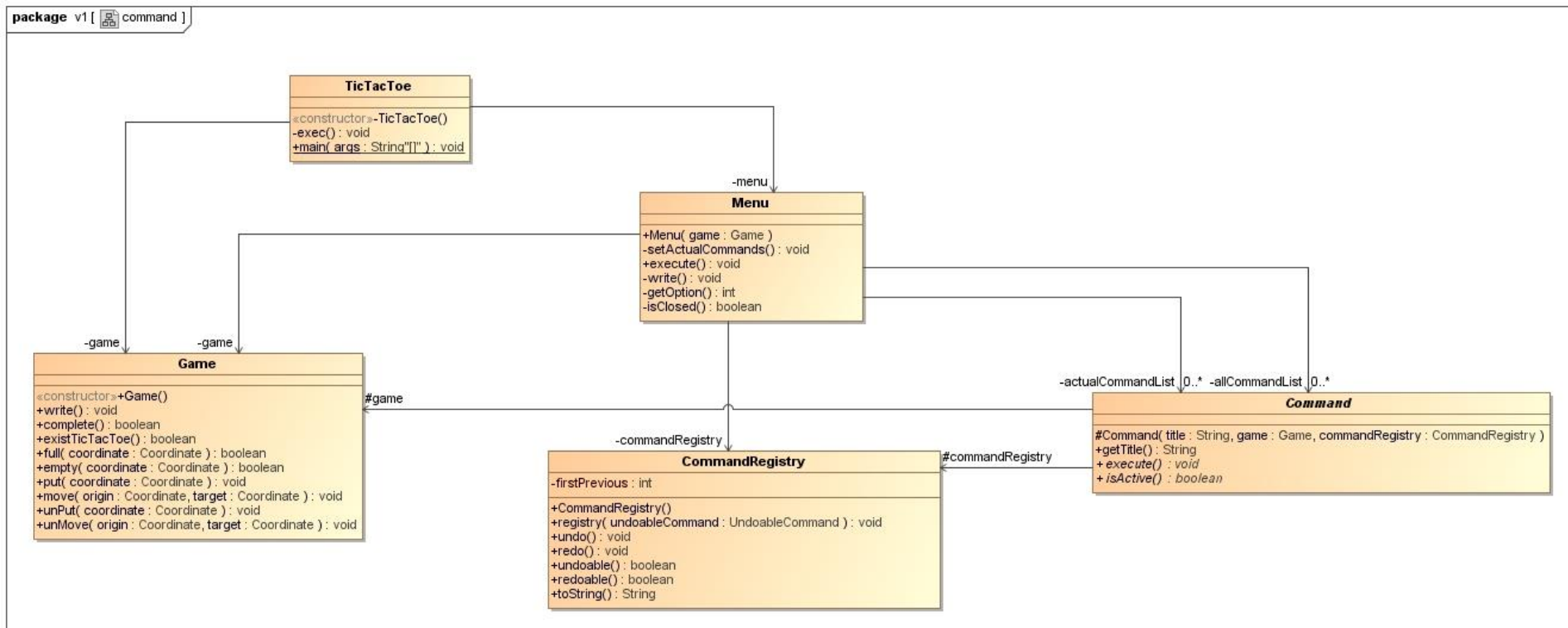


## 6. Juego de las Tres En Raya

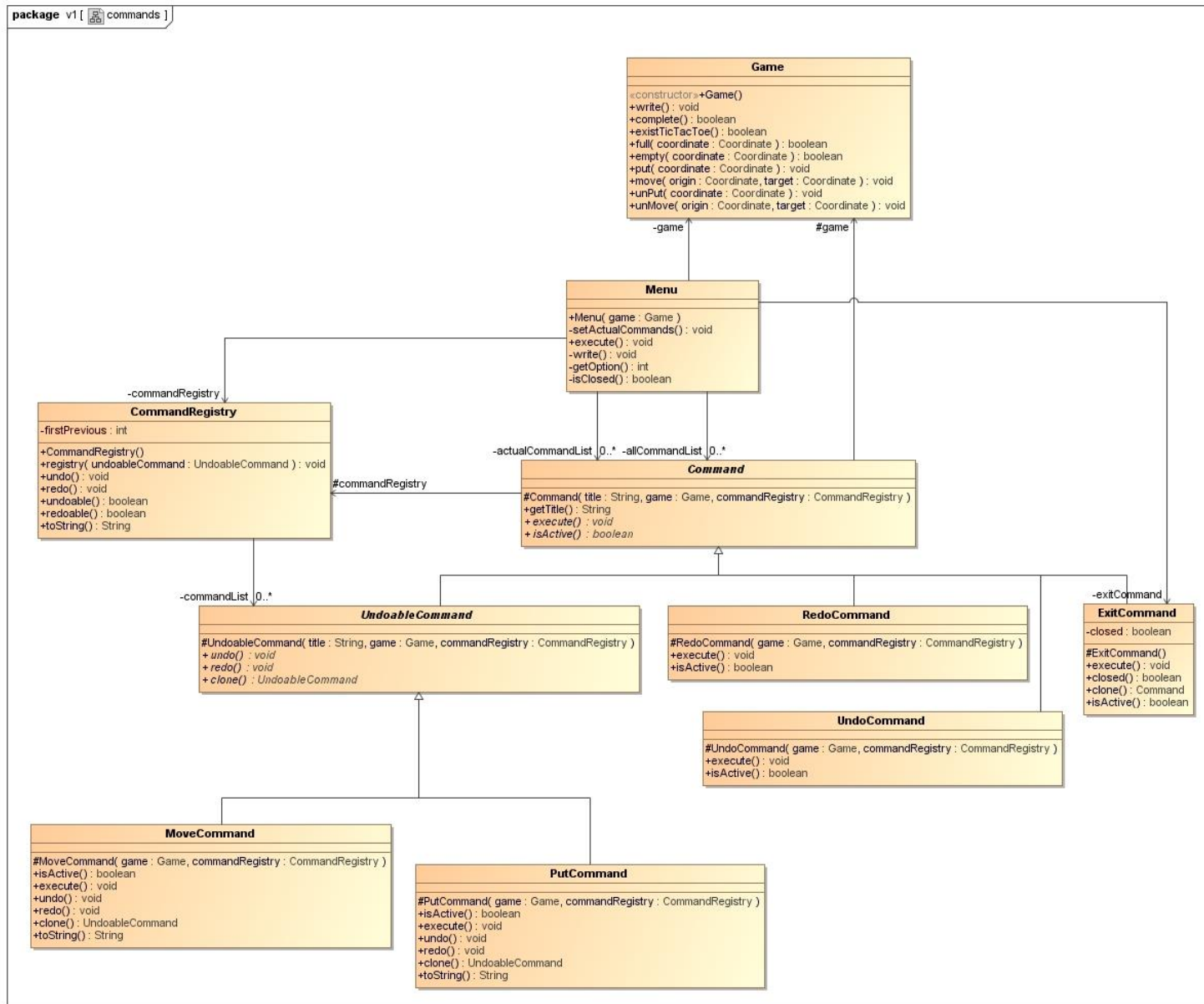




## 6. Juego de las Tres En Raya



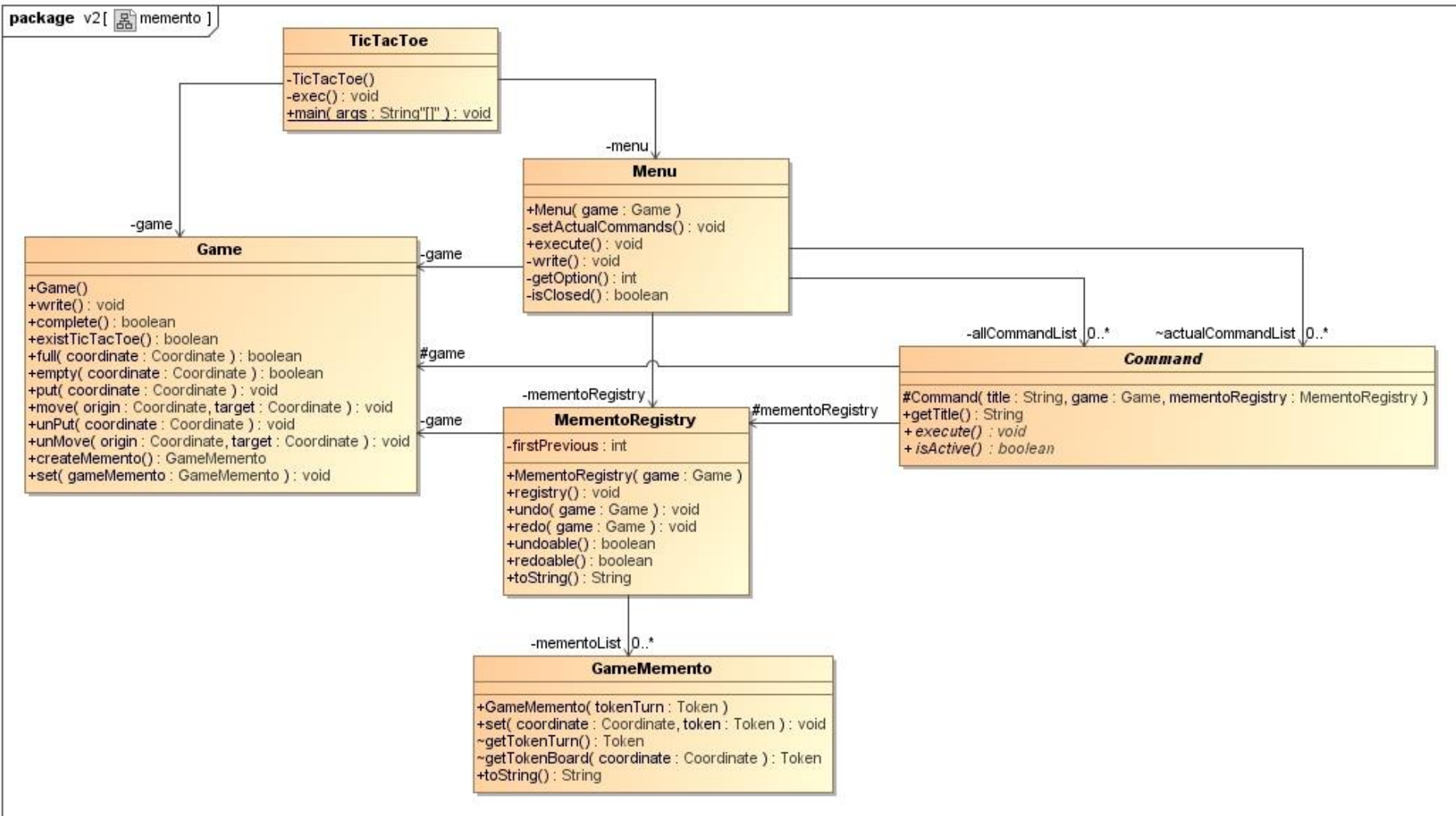
# 7. Juego de las Tres En Raya



## 7. Juego de las Tres En Raya

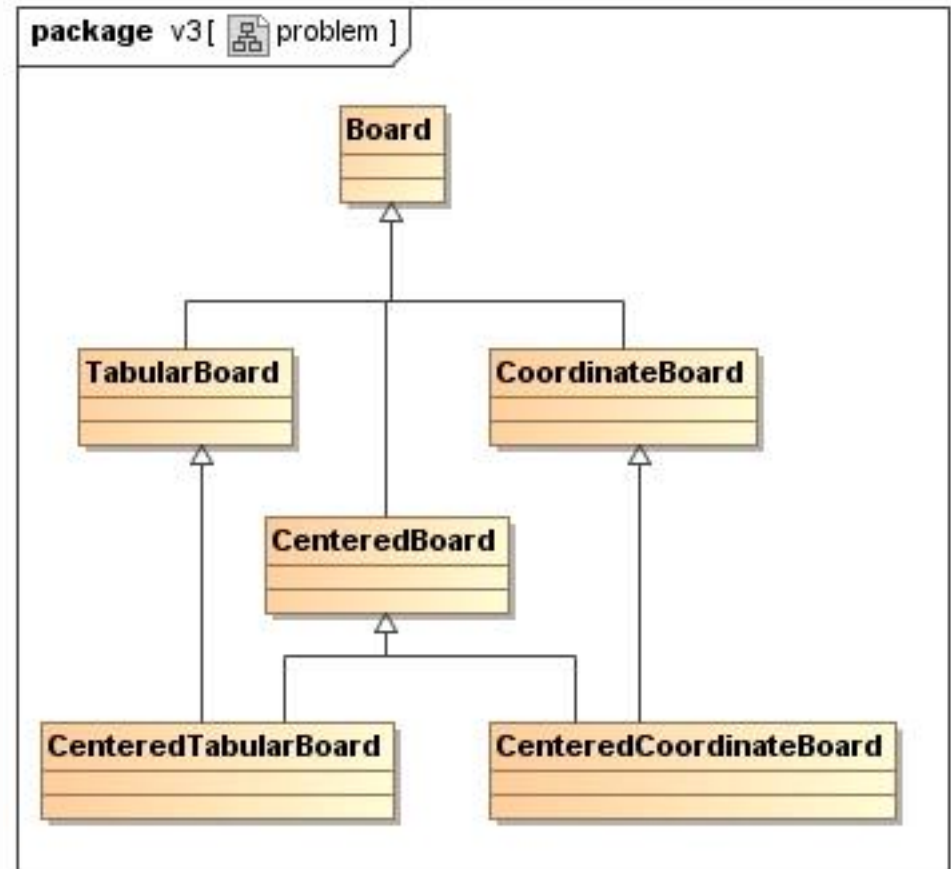
- **Problema.** Se desea deshacer/rehacer sin la re-ejecución de las opciones.
  - Especificación de interfaces de los objetos: *Memento*
- **Intención.** Sin violar la encapsulación, captura y externaliza el estado interno de un objeto de tal forma que el objeto puede ser restaurado a este estado después
- **Interrelación.** *Singleton*
- **Propósito.**
  - ~~Composite vs Memento.~~ Combinables
- **Rediseño.** Dependencias de las representaciones o implementaciones de los objetos
- **Variación.** Qué información privada es almacenada fuera del objeto y cuándo

## 7. Juego de las Tres En Raya



## 7. Juego de las Tres En Raya

- **Enunciado.** Se desea:
  - Estudiar dos posibles implementaciones basadas en:
    - *Tabular*: Almacenar las fichas en las 3x3 casillas del tablero
    - *Coordenadas*: Almacenar las coordenadas de las fichas de cada jugador
  - Dar cabida a nuevas clases de tablero: el que se inicializa con una ficha en el centro, ...



## 7. Juego de las Tres En Raya

- **Problema.** Se desea desacoplar la implantación del tablero para realizar una clasificación de éstos sin “repetición”.
  - Delegación: *Bridge*
- **Intención.** *Desacopla una abstracción de su implementación de tal forma que las dos pueden variar independientemente*
- **Interrelación.** *Abstract Factory*
- **Propósito.**
  - ~~Adapter vs Bridge~~. No tengo clases en librerías, ...
- **Rediseño.** Dependencias de las representaciones o implementaciones de los objetos
- **Variación.** La implementación de un objeto



# 7. Juego de las Tres En Raya

package v3[  bridge ]